Advanced Threat Detection Using Machine Learning for URL Safety

Team Members:

1. Sai Aparanji Nemmani - nsaiaparanji@vt.edu
2. Haritha Injam - iharitha@vt.edu
3. Sushma Shivani Nukala - sushmashivani@vt.edu

Introduction

In today's digitally interconnected world, malicious URLs represent a growing security challenge. These URLs are increasingly used as vectors to distribute malware, steal sensitive data, and manipulate users into compromising their systems. The project, titled *Advanced Threat Detection Using Machine Learning for URL Safety*, seeks to develop an advanced system that leverages machine learning to proactively detect and block these harmful URLs. The proposed solution will focus on accurately identifying suspicious URLs in real-time, preventing access to malicious content and safeguarding users from a wide range of cyber-attacks initiated through compromised URLs.

The rapid spread of harmful URLs across the internet has become a critical threat to cybersecurity. These URLs serve as conduits for malware, phishing schemes and other attacks that compromise the integrity of user data and online safety. Common cyber-attacks associated with malicious URLs—such as malware distribution, phishing and web page defacement—leverage various vulnerabilities to deceive and exploit individuals, businesses and public websites.

Malware can be discreetly distributed through URLs embedded within seemingly legitimate web pages or online advertisements. When an unsuspecting user clicks on a compromised link, malware may be silently downloaded onto their device, allowing the attacker unauthorized access to sensitive information. Phishing, another prevalent threat, is a form of social engineering that aims to trick individuals into divulging personal data by impersonating trusted entities. Attackers often send deceptive emails, chat messages, or social media posts that include links to fake websites designed to harvest user credentials and other private information. Defacement attacks, meanwhile, target the appearance and content of websites, often undermining credibility or displaying unauthorized messages. Such attacks involve altering or replacing legitimate web content with malicious messages, images, or redirects, eroding user trust and damaging the reputation of the affected website.

The sheer volume of online interactions today makes it essential to have robust defenses in place. In 2023, there were more than 3.4 million phishing attempts globally, many of which were linked to malicious URLs. This project aims to enhance security measures by using machine learning models that continuously adapt to the evolving tactics of cybercriminals. By leveraging advanced machine learning algorithms, the system will analyze URLs to distinguish between benign and malicious links, providing a proactive defense mechanism for identifying and blocking threats.

Through this initiative, the project not only reacts to existing threats but also predicts emerging patterns, offering a proactive layer of protection for users and organizations alike. This project proposes a sophisticated URL classification model designed to detect and prevent these types of attacks, enhancing online security and strengthening cybersecurity practices by mitigating the risks associated with malicious URLs.

Problem Statement

The exponential growth of the internet has introduced new security challenges, and malicious URLs represent one of the most pressing issues in the cybersecurity landscape. With cybercriminals increasingly turning to sophisticated tactics, existing defenses are often insufficient. A 2020 study by Cybersecurity Ventures forecasts that cybercrime costs could reach $10.5 trillion annually by 2025, a significant portion of which will stem from malicious URLs. The ability to detect and prevent these threats in real time has become a critical requirement for modern cybersecurity systems.

The challenge lies in the versatility of malicious URLs. Cybercriminals are adept at using obfuscation techniques to disguise harmful URLs as legitimate ones. Common methods include the use of seemingly trusted domain names with subtle modifications (e.g., “paypa1.com” instead of “paypal.com”) or the employment of URL shorteners to mask dangerous content. These techniques allow attackers to bypass static security measures, such as blacklists, which can only block URLs after they have been identified as harmful.

This project seeks to go beyond static defense mechanisms by developing a dynamic, machine-learning-based system to predict malicious URLs based on their inherent features. Unlike traditional methods that focus solely on known threats, this system will analyze a wide range of URL characteristics—including length, domain patterns, special characters and the use of IP addresses instead of domain names—to detect emerging threats in real time. This adaptive approach will not only improve detection accuracy but also allow the system to keep pace with the constantly evolving tactics of cybercriminals.

By leveraging machine learning algorithms, this project will develop a more scalable and efficient solution that can identify threats early, minimizing the risk of phishing, malware distribution and defacement attacks. The goal is to build a robust, adaptive system that provides real-time protection against the ever-changing landscape of malicious URLs.

Dataset Description

This project utilizes a *Malicious URLs* dataset from Kaggle, consisting of 651,191 samples categorized into four features: benign, defacement, phishing, and malware. This dataset provides a robust foundation for training and evaluating a detection system that differentiates between benign and malicious URLs. Samples were curated from multiple sources, including ISCX-URL-2016, the Malware Domain Blacklist, Phishtank, PhishStorm, and a GitHub repository by Faizan

for additional benign URLs. URLs from each source were initially organized in separate data frames and then merged to retain only the URL samples and their corresponding feature labels, resulting in a comprehensive labeled dataset for threat detection models.

The dataset comprises the following URL categories:

- **Benign URLs**: 4,28,103
- **Defacement URLs**: 96,457
- **Phishing URLs**: 94,111
- **Malware URLs**: 32,520

This class distribution represents a variety of malicious types alongside a significant volume of benign samples, allowing for comprehensive model training and testing. The distribution across these categories is shown in the following Bar Chart.

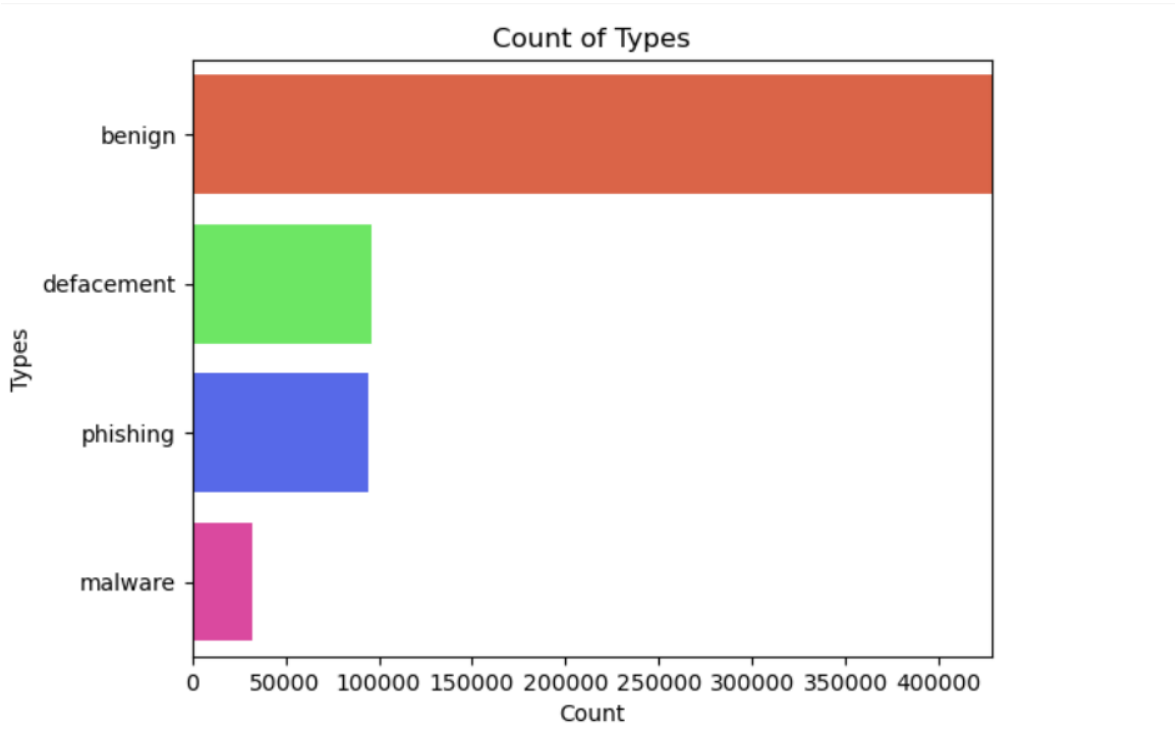


Figure 1: Distribution of the target('type') of the dataset

Preprocessing Steps:

1. **Feature Extraction:** From the "URL" attribute in the dataset, 150 features have been extracted to comprehensively capture its structural and statistical characteristics. These include metrics like the total URL length, domain structure, subdomains, top-level domain (TLD), and the full domain string. Additional features, such as character entropy and counts of special characters, offer deeper insights into the composition of each URL, making them valuable for detailed analysis and understanding.

Table 1: Extracted Features of the Dataset

No.	Feature Name	Description
1	type	Type of URL (e.g., benign, phishing, malware, etc.).
2	Category	Encoded version of the type column.
3	url_len	Length of the entire URL string.
4	domain	Extracted domain of the URL.
5	@	Count of the @ symbol in the URL.
6	?	Count of the ? symbol in the URL.
7	-	Count of the - symbol in the URL.
8	=	Count of the = symbol in the URL.
9	.	Count of the . symbol in the URL.
10	#	Count of the # symbol in the URL.
11	%	Count of the % symbol in the URL.
12	+	Count of the + symbol in the URL.
13	$	Count of the $ symbol in the URL.
14	!	Count of the ! symbol in the URL.
15	*	Count of the * symbol in the URL.

16	,	Count of the , symbol in the URL.
17	//	Count of the // sequence in the URL.
18	https	Indicates if the URL uses HTTPS (1) or not (0).
19	digits	Count of numeric characters in the URL.
20	letters	Count of alphabetic characters in the URL.
21	Shortening_Service	Indicates if the URL uses a URL shortening service (e.g., bit.ly).
22	Abnormal_URL	Indicates if the URL structure is abnormal (e.g., domain not matching URL).
23	is_ip	Indicates if the URL contains an IP address instead of a domain.
24	url_path	Extracted path component of the URL.
25	url_path_len	Length of the path component of the URL.
26	count_dir_in_url_path	Number of directories in the URL path (based on /).
27	first_dir_len	Length of the first directory in the URL path.
28	subdomain	Extracted subdomain of the URL.
29	tld	Top-level domain (TLD) of the URL.
30	fld	Full-length domain of the URL.
31	subdomain_len	Length of the subdomain.

32	tld_len	Length of the TLD.
33	fld_len	Length of the full-length domain.
34	url_puncs	Count of punctuation characters in the URL.
35	url_len_q	Quartile rank of the URL length.
36	fld_len_q	Quartile rank of the full-length domain length.
37	pc_alphas	Percentage of alphabetic characters in the URL.
38	pc_digits	Percentage of numeric characters in the URL.
39	pc_puncs	Percentage of punctuation characters in the URL.
40	hostname_len	Length of the hostname of the URL.
41	has_port_no	Indicates if the URL contains a port number.
42	number_of_params	Number of query parameters in the URL.
43	number_of_fragments	Number of fragments (e.g., after #) in the URL.
44	has_client	Indicates if the URL contains the keyword "client".
45	has_admin	Indicates if the URL contains the keyword "admin".
46	has_server	Indicates if the URL contains the keyword "server".
47	has_login	Indicates if the URL contains the keyword "login".
48	entropy_of_url	Shannon entropy of the entire URL string.

49	entropy_of_subdomain	Shannon entropy of the subdomain.
50	entropy_of_domain	Shannon entropy of the domain.
51	entropy_of_tld	Shannon entropy of the TLD.
52	entropy_of_fld	Shannon entropy of the full-length domain.
53	num_suspicious_punctuations	Count of suspicious punctuation characters.
54	contains_admin_terms	Indicates if the URL contains admin-related terms.
55	contains_login_terms	Indicates if the URL contains login-related terms.
56	contains_server_terms	Indicates if the URL contains server-related terms.
57	contains_client_terms	Indicates if the URL contains client-related terms.
58	digit_to_alphabet_ratio	Ratio of numeric characters to alphabetic characters in the URL.
59	special_character_to_length_ratio	Ratio of special characters to total URL length.
60	query_to_domain_ratio	Ratio of query string length to domain length.
61	num_subdomains	Number of subdomains in the URL.
62	longest_subdomain_length	Length of the longest subdomain.
63	domain_reputation_flag	Indicates if the domain is associated with low reputation.
64	suspicious_keyword_count	Count of suspicious keywords in the URL.
65	hexadecimal_pattern_count	Count of hexadecimal patterns in the URL.

66	contains_suspicious_terms	Indicates if the URL contains suspicious terms.
67	has_http	Indicates if the URL starts with "http".
68	path_entropy	Shannon entropy of the URL path.
69	domain_to_url_ratio	Ratio of domain length to total URL length.
70	path_to_url_ratio	Ratio of path length to total URL length.
71	query_to_url_ratio	Ratio of query length to total URL length.
72	percent_special_chars	Percentage of special characters in the URL.
73	percent_uppercase	Percentage of uppercase letters in the URL.
74	percent_lowercase	Percentage of lowercase letters in the URL.
75	url_length	Overall length of the URL.
76	domain_length	Overall length of the domain.
77	num_directories	Number of directories in the URL path.
78	num_query_params	Number of query parameters in the URL.
79	is_long_url	Indicates if the URL length exceeds a predefined threshold.
80	high_entropy_url	Indicates if the URL has high entropy.
81	has_special_chars	Indicates if the URL contains special characters.
82	len_x_digits	Length divided by the number of digits in the URL.

83	len_x_letters	Length divided by the number of letters in the URL.
84	path_depth_ratio	Depth of the URL path relative to its length.
85	entropy_len_digits	Entropy normalized by the number of digits.
86	entropy_len_puncs	Entropy normalized by the number of punctuation characters.
87	len_div_digits	Length divided by the number of digits.
88	letters_div_digits	Ratio of letters to digits.
89	puncs_x_digits	Number of punctuation multiplied by the number of digits.
90	len_div_puncs	Length divided by the number of punctuation characters.
91	subdomain_x_tld	Subdomain length multiplied by TLD length.
92	has_multiple_tlds	Indicates if the URL contains multiple TLDs.
93	is_generic_tld	Indicates if the URL contains a generic TLD.
94	has_www	Indicates if the URL contains "www".
95	is_numeric_tld	Indicates if the TLD is numeric.
96	count_slashes	Count of slashes (/) in the URL.
97	count_underscore	Count of underscores (_) in the URL.
98	count_uppercase	Count of uppercase letters in the URL.

99	count_lowercase	Count of lowercase letters in the URL.
100	word_count	Total number of words in the URL.
101	avg_word_len	Average word length in the URL.
102	contains_verify	Indicates if the URL contains the word "verify".
103	contains_secure	Indicates if the URL contains the word "secure".
104	contains_bank	Indicates if the URL contains the word "bank".
105	contains_account	Indicates if the URL contains the word "account".
106	contains_free	Indicates if the URL contains the word "free".
107	contains_offer	Indicates if the URL contains the word "offer".
108	is_long_subdomain	Indicates if the subdomain length exceeds a predefined threshold.
109	url_len_rolling_mean	Rolling mean of URL lengths in the dataset.
110	url_len_rolling_std	Rolling standard deviation of URL lengths.
111	letters_rolling_mean	Rolling mean of letter counts in the dataset.
112	letters_rolling_std	Rolling standard deviation of letter counts.
113	digits_rolling_mean	Rolling mean of digit counts in the dataset.
114	digits_rolling_std	Rolling standard deviation of digit counts.
115	url_puncs_rolling_mean	Rolling mean of punctuation counts in the dataset.

116	url_puncs_rolling_std	Rolling standard deviation of punctuation counts.
117	url_len_rank	Rank of the URL length within the dataset.
118	letters_rank	Rank of the number of letters in the URL.
119	digits_rank	Rank of the number of digits in the URL.
120	url_puncs_rank	Rank of the number of punctuation characters in the URL.
121	has_suspicious_subdomain	Indicates if the subdomain is suspicious.
122	is_commercial_tld	Indicates if the TLD is commercial (e.g., .com).
123	is_nonprofit_tld	Indicates if the TLD is non-profit (e.g., .org).
124	is_country_code_tld	Indicates if the TLD is a country code (e.g., .uk).
125	has_multiple_fragments	Indicates if the URL contains multiple fragments.
126	has_multiple_parameters	Indicates if the URL contains multiple parameters.
127	has_admin_and_suspicious	Indicates if the URL contains both admin-related and suspicious terms.
128	has_login_and_high_entropy	Indicates if the URL contains "login" and has high entropy.
129	longest_path_segment	Length of the longest segment in the URL path.
130	numeric_subdomains_count	Count of numeric subdomains.
131	capitalization_ratio	Ratio of uppercase letters to total letters in the URL.

132	repetitive_char_count	Count of consecutive repetitive characters in the URL.
133	is_non_standard_tld	Indicates if the TLD is non-standard or uncommon.
134	spam_keyword_count	Count of spam-related keywords in the URL.
135	brand_mismatch_flag	Indicates if the URL contains mismatched branding terms.
136	has_suspicious_extension	Indicates if the URL has a suspicious file extension.
137	domain_is_english	Indicates if the domain contains English words.
138	is_popular_domain	Indicates if the domain belongs to a popular/trusted list.
139	redirect_count	Number of redirects in the URL.
140	contains_time_pattern	Indicates if the URL contains time-related patterns (e.g., year, month).
141	domain_similarity_score	Similarity score between domain and subdomain using sequence matching.
142	path_segment_entropy	Shannon entropy of the path segments in the URL.
143	external_scripts	Indicates if the URL includes external scripts.
144	has_unusual_subdomain	Indicates if the subdomain contains unusual patterns (e.g., --).
145	contains_base64	Indicates if the URL contains Base64-encoded strings.
146	median_word_length_path	Median length of words in the URL path.

147	character_diversity_index	Ratio of unique characters to total characters in the URL.
148	max_consecutive_symbols	Maximum number of consecutive special symbols in the URL.
149	homoglyph_count	Count of homoglyph characters (e.g., 0, O, 1, l) in the URL.
150	keyword_context_entropy	Shannon entropy of keyword occurrences in the URL's path.

2. **Data Preprocessing Steps**

- Excluding Textual Features: Textual attributes such as URL, domain, subdomain, TLD, FLD, and URL path have been removed from the dataset after extracting all relevant information.
- Handling Missing Values: The first_dir_len column has 14 missing values due to certain URLs lacking '/' to separate directories. Since these instances are minimal compared to the dataset size, the missing values were imputed with 0.
- Encoding the Target Attribute: The target attribute, originally containing class labels as strings, has been encoded into integers as follows: "benign" → 0, "defacement" → 1, "phishing" → 2, "malware" → 3.
- Discretizing Numerical Features: Features like url_len and fld_len, which show a strong correlation with the target, were discretized into categories such as "Short," "Medium," "Long," and "Very Long." One-hot encoding was then applied to these categories.
- Encoding Other Categorical Features: Boolean variables were converted to binary values, represented as 0 and 1.
- Tokenization for CNN: The original URLs were tokenized at the character level to prepare them for processing by the CNN model.

3. **Feature Selection:** To identify the most relevant features for model performance, two different feature selection techniques were applied to the dataset:

1. Random Forest Feature Importance Ranking: A Random Forest model was employed to determine feature importance scores based on how influential each attribute was in the model’s predictive process. This technique leverages an ensemble of decision trees, assigning importance based on the frequency and impact of features across all trees. Features with higher scores were prioritized, as they represent attributes that are consistently valuable across the dataset. This step provided a robust ranking that helped refine the feature set further, improving efficiency in model training and reducing the risk of overfitting.

2. Correlation Analysis with Target Variable: A correlation analysis was conducted to assess the relationship between individual features and the target variable. By calculating correlation scores, we identified features that exhibit significant alignment with the target. These correlations help isolate attributes that are more predictive, allowing for a more streamlined dataset by retaining only those features that contribute meaningful information.

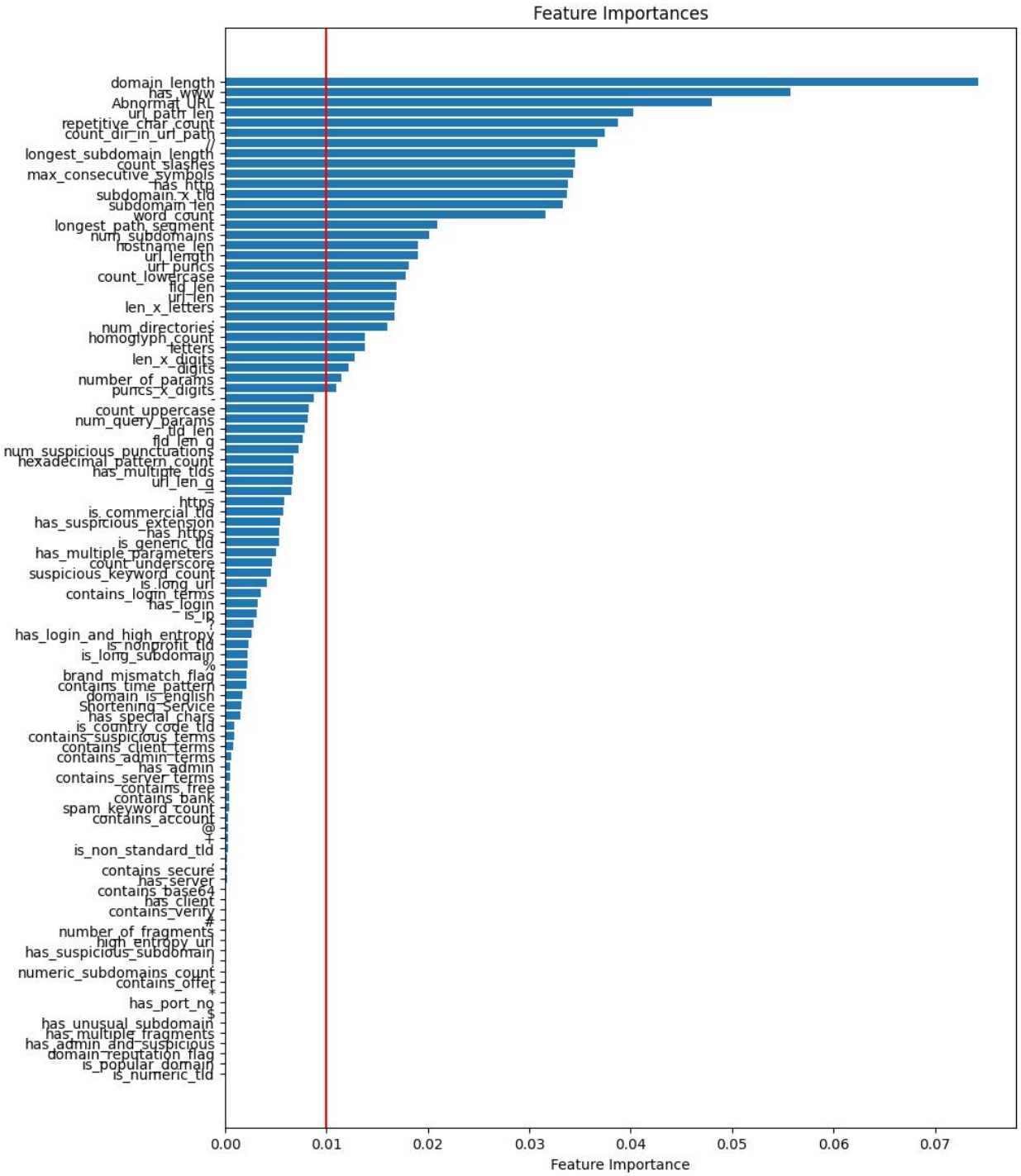


Figure 2: Random Forest Feature Importance

Methods and Models:

1. **Analyzing ML algorithms:** To effectively assess various machine learning models, the dataset was split into an 80/20 train-test configuration, ensuring a balanced approach to training and validation. A range of classifiers was then applied, each offering distinct strengths suited to different aspects of the dataset. The algorithms tested included:

- **Decision Tree Classifier**: Decision trees make decisions by splitting the data based on features and creating a tree-like structure of decisions. Decision trees can capture complex decision boundaries, making them suitable for datasets with intricate patterns. They can handle both numerical and categorical features, making them versatile for URL analysis.
- **Random Forest Classifier**: Random Forest is an ensemble learning method that constructs a multitude of decision trees and merges their outputs to improve generalization and robustness. Random Forests are effective in handling high-dimensional data and are less prone to overfitting. They can provide feature importance, helping to identify which features contribute the most to classification.
- **AdaBoost Classifier**: AdaBoost is an ensemble learning technique that combines multiple weak classifiers to create a strong classifier. It is useful when there are weak learners in the dataset. It can adapt well to complex datasets and is less likely to overfit. It may be employed to improve the performance of base classifiers.
- **K-Neighbors Classifier**: KNN classifies a data point based on the majority class of its k-nearest neighbors in the feature space. KNN is simple and intuitive. It can capture local patterns in the data. However, it might struggle with high-dimensional data and can be sensitive to irrelevant features.
- **SGD Classifier**: SGD is an optimization algorithm that is used in conjunction with linear classifiers, such as Support Vector Machines and logistic regression. SGD classifiers are efficient for large-scale datasets and high-dimensional feature spaces. They can handle sparse data and are suitable for online learning.
- **Extra Trees Classifier**: Extra Trees, or Extremely Randomized Trees, is an ensemble learning method similar to Random Forests but introduces additional randomness in the feature selection process. Extra Trees can be robust to noisy data and are computationally efficient. They are suitable for handling high-dimensional datasets.
- **Gaussian Naive Bayes (GaussianNB)**: Naive Bayes is based on Bayes' theorem and assumes independence between features. Naive Bayes classifiers are simple and computationally efficient. They can work well with high-dimensional data but might struggle if the independence assumption is violated.
- **Support Vector Classifier**: Support Vector Classification (SVC) is a type of support vector machine (SVM) that is used for classification tasks. SVMs, including SVC, are popular machine learning models that excel in both linear and non-linear classification. They work by finding the optimal hyperplane that separates data points of different classes in feature space. In URL detection, the feature space can be high-dimensional, considering various URL attributes. SVM can handle high-dimensional data effectively and is less prone to overfitting.

- **Convolutional Neural Network**: CNN is a type of deep learning model designed for processing structured grid data, such as images or sequences. For URL analysis, a character-level CNN can learn hierarchical representations of the characters in URLs, capturing complex patterns. It's effective when there are intricate relationships between characters.The architecture of the 1D-Character level CNN is demonstrated in Figure 1

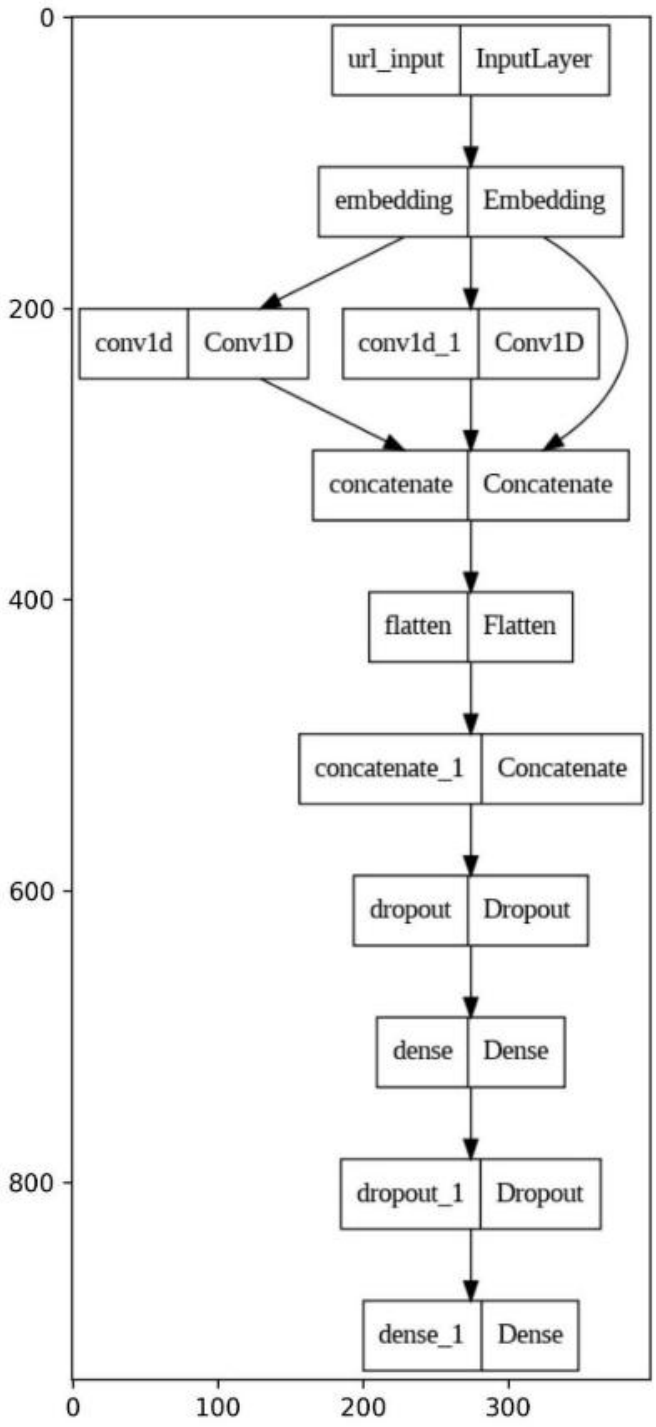


Figure 3: Architecture of 1D-CNN model

Evaluating Models:

To measure and compare model performance, a range of evaluation metrics was employed, offering a multi-faceted view of each model's strengths and weaknesses:

- **Accuracy**: Reflects the percentage of correctly classified instances, providing a general performance overview.
- **Precision and Recall**: Precision highlights the model's ability to minimize false positives, while recall assesses its capability to detect true positives, crucial for imbalanced classes.
- **F1 Score**: The harmonic mean of precision and recall, is useful for balancing these metrics, particularly in cases with class imbalances.
- **Receiver Operating Characteristic (ROC) Curve** and **Area Under the Curve (AUC)**: These metrics measure the model's ability to differentiate between classes. AUC provides a single score that summarizes the model's performance across various threshold levels.
- **Confusion Matrix**: A breakdown of true positives, false positives, true negatives, and false negatives, offering a clear view of prediction errors for each class.

Results:

The result analysis in terms of precision, recall, accuracy and F1 score is given below:

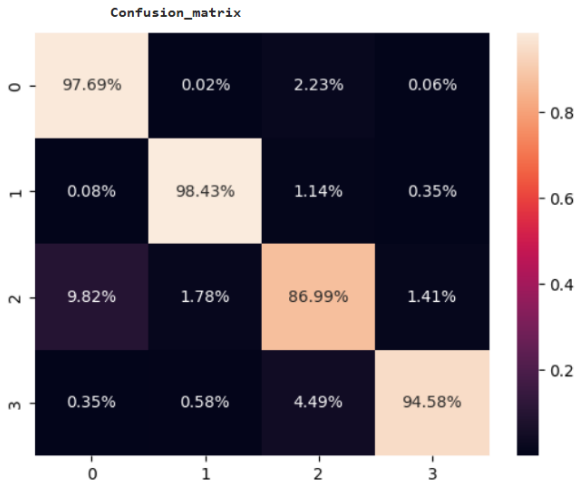
Table 2: Result Analyses of the Developed Models

Feature Selection Method	**Classifier**	**Precision**	**Recall**	**F1 score**	**Accuracy (%)**
Selecting All Features	Decision Tree	0.96	0.96	0.96	96.10
	Random Forest	0.97	0.97	0.97	97.44
	Adaboost	0.78	0.83	0.79	82.67
	KNN	0.95	0.95	0.95	94.90
	SGD	0.80	0.82	0.80	82.31

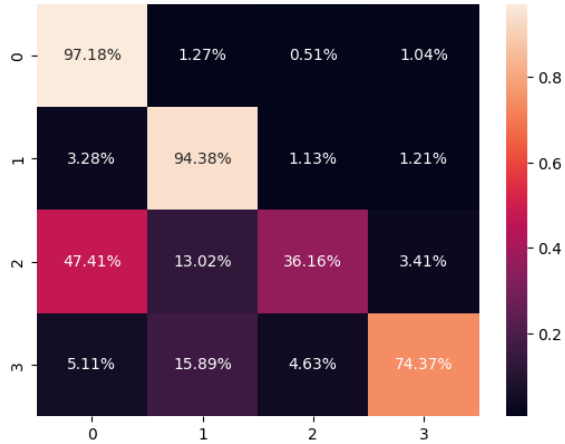
	ExtraTree	0.97	0.97	0.97	97.34
	Gaussian NB	0.76	0.79	0.75	79.19
	SVC	0.75	0.73	0.66	72.84
Selecting Features by correlation with the target	Decision Tree	0.82	0.83	0.82	82.53
	Random Forest	0.87	0.87	0.85	86.80
	Adaboost	0.74	0.65	0.68	64.86
	KNN	0.83	0.84	0.82	84.09
	SGD	0.72	0.75	0.72	75.15
	ExtraTree	0.86	0.87	0.85	86.59
	Gaussian NB	0.74	0.78	0.74	77.79
	SVC	0.79	0.82	0.76	81.72
Selecting Features by Random Forest Feature Importance	Decision Tree	0.93	0.93	0.93	93.25
	Random Forest	0.95	0.95	0.95	94.87
	Adaboost	0.81	0.83	0.82	83.24
	KNN	0.93	0.94	0.94	93.65
	SGD	0.79	0.81	0.76	81.49
	ExtraTree	0.95	0.95	0.95	94.81
	Gaussian NB	0.74	0.78	0.74	78.26
	SVC	0.79	0.82	0.76	81.76

	1D CNN	0.97	0.97	0.97	97.67

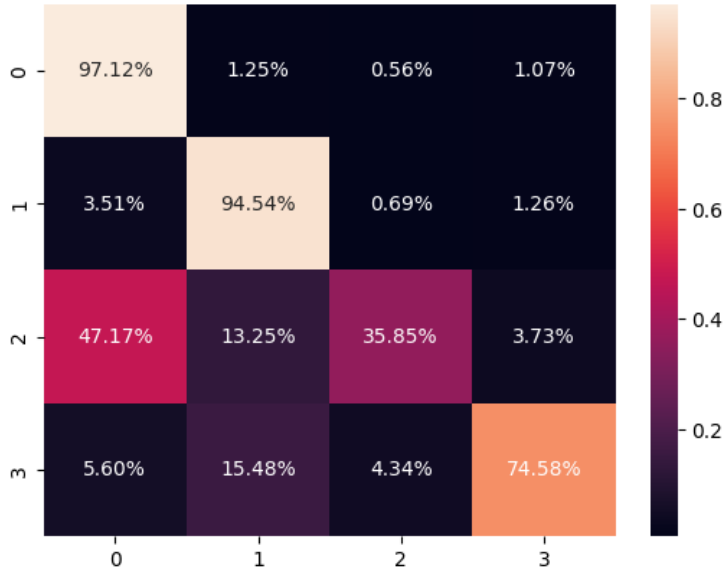
The analysis highlights the consistently strong performance of the Random Forest classifier particularly when coupled with suitable feature selection methods. Feature selection methods play a significant role in classifier performance, with feature importance-based approaches demonstrating superiority. The confusion matrices of the Random Forest model are provided for various feature selection scenarios, revealing its effectiveness in classifying malicious URLs.



a) Selecting all Features

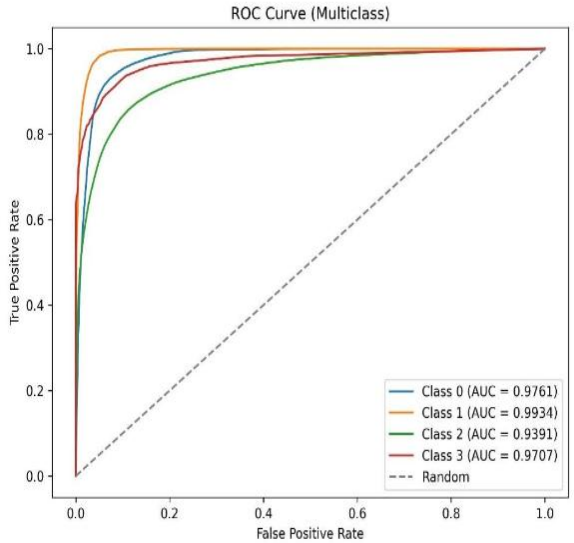


b) Selecting Features by correlation

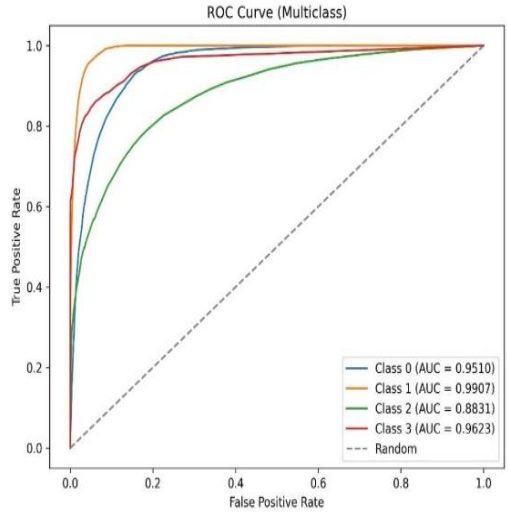


c) Selecting Features by Random Forest Feature Importance

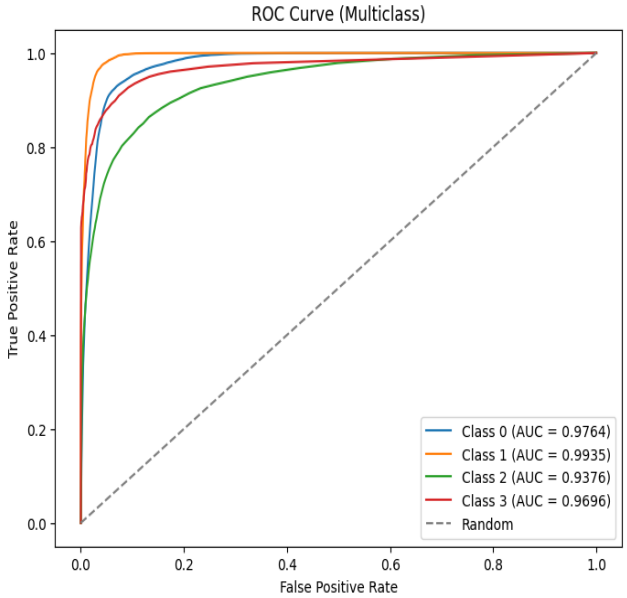
Figure 4: Confusion matrices of the Random Forest Classifier



a) Selecting all features



b) Selecting Features by correlation



b) Selecting Features by Random Forest Feature Importance

Figure 5: ROC-AUC of the Random Forest Classifier

It is also evident from the table that the character level 1D CNN classifier exhibits exceptional results suggesting that deep learning approaches could be effective for the detection task. The confusion matrix of the character level CNN is given below:

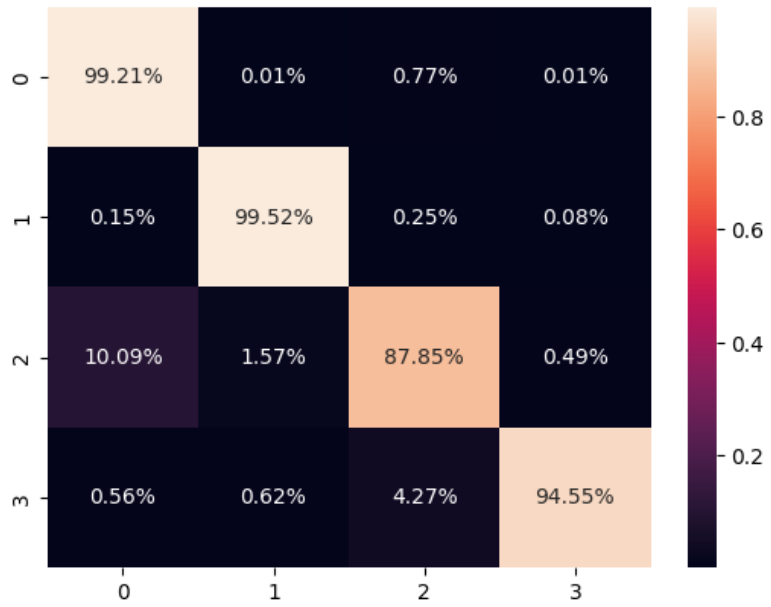


Figure 6: Confusion matrices of the character level CNN

The ROC-AUC of the character level CNN is given below:

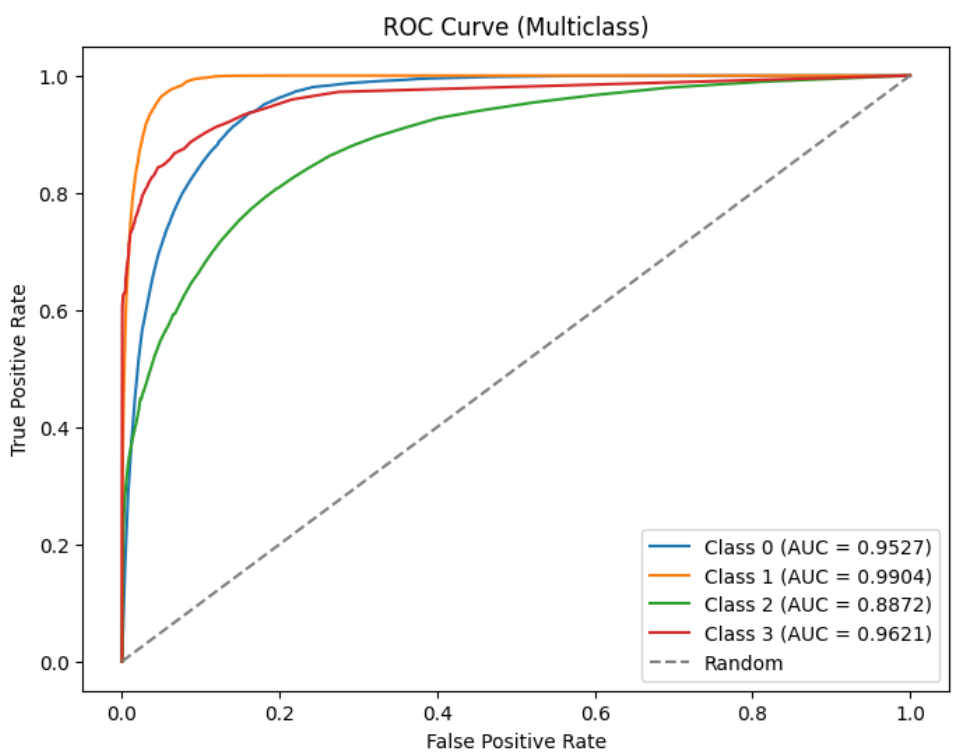


Figure 7: ROC-AUC of the character level CNN

The confusion matrices and the ROC-AUC curves of the rest of the models can be found in the Appendix section. However, studying the confusion matrices we can see that even if the models perform better for label 0 and 1 which are classes such as ‘benign’ and ‘defacement’, they all collectively falter in correctly classifying label 2 or ‘phishing’ attacks. The ROC curves also show comparatively less AUC score in class label 2 than other classes.

Overall, the results analysis showcases the performance of various classifiers with different feature selection methods in the task of Malicious URL Detection. It provides valuable insights into the strengths and weaknesses of different combinations. The key findings are summarized as follows:

1. The analysis reveals that Random Forest and ExtraTree classifiers consistently deliver strong performance across multiple feature selection methods. These classifiers, in conjunction with appropriate feature selection, demonstrate high Precision, Recall, F1 scores, and Accuracy.

2. The choice of feature selection method plays a crucial role in classifier performance. Feature importance-based methods, such as those using Random Forest Feature Importance, consistently outperform other feature selection techniques.

3. In contrast, Adaboost and Gaussian NB classifiers exhibit comparatively weaker performance suggesting the need for further optimization or alternative feature engineering strategies.

4. The character-level 1D CNN classifier showcases exceptional results, hinting at the effectiveness of deep-learning approaches for malicious URL detection.

5. A closer examination of the confusion matrices and ROC curves indicates a collective challenge in correctly classifying label 2 which represents 'phishing' attacks. Despite the strong performance in classifying labels 0 ('benign') and 1 ('defacement') there is room for improvement in handling phishing attacks.

Conclusion:

This project can be successfully developed and implemented as an effective system for identifying and blocking malicious URLs. By leveraging a combination of traditional machine learning models and deep learning techniques, robust results can be achieved in classifying URLs as benign or malicious. This solution has a significant impact, enhancing online security, protecting users from cyber threats, and contributing to a safer online environment. The insights/inferences from this project can be summarized as below:

- Feature extraction, including various characteristics of URLs such as length, entropy, and counts of specific characters, contributes to building a robust model for detecting malicious URLs. Domain-related information and URL structure are crucial in capturing patterns indicative of phishing, malware, and defacement attacks. The choice of features aligns with the understanding that the landscape of malicious URLs is continually

evolving. By leveraging features related to URL structure and content, the model can adapt to new techniques employed by cybercriminals.

- The project emphasizes the importance of feature selection for model performance. Features like URL length, domain, subdomain, and various counts (characters, special characters) play a crucial role in distinguishing between benign and malicious URLs. The Random Forest feature importance technique highlights specific features that contribute significantly to the reduction in impurity across decision trees. This emphasizes the need for focusing on relevant features to improve model interpretability and avoid overfitting.
- The exceptional performance of the 1D CNN classifier suggests that deep learning, especially at the character level, is effective in capturing intricate patterns within URLs. This insight implies that the hierarchical representations learned by a CNN can discern complex relationships between characters.
- We can observe a collective challenge in correctly classifying phishing attacks (label 2). This insight indicates a potential area for improvement in the model's ability to identify and differentiate phishing URLs effectively.

In summary, the impact of features extracted from the data underscores the importance of thoughtful feature selection and the effectiveness of deep learning in addressing the challenges of malicious URL detection. The insights provide guidance for refining the model, improving its capability to handle evolving threats, and enhancing its practical application in real-world scenarios.

References:

[1] Ma,Justin, Saul,Lawrence, Savage,Stefan, and Voelker,Geoffrey. (2009). URL Reputation. UCI Machine Learning Repository. https://doi.org/10.24432/C5H89Q.

Individual Contributions:

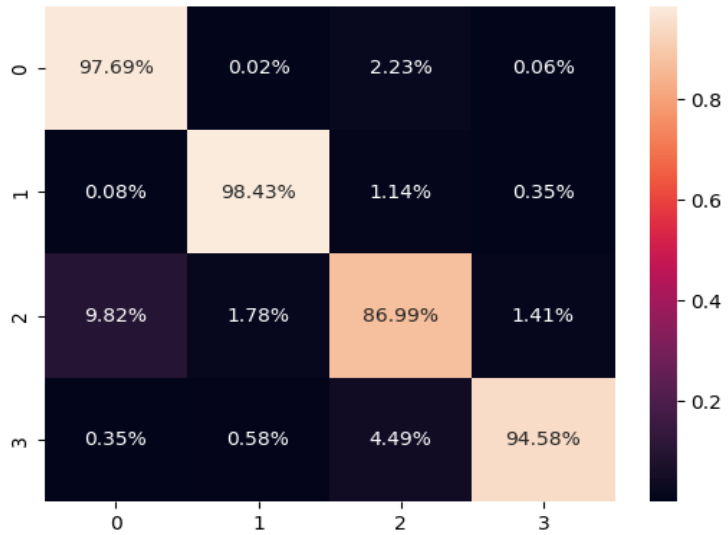
Member 1: Sushma Shivani Nukala	**Task 1:** Participated in feature extraction by focusing on advanced characteristics, including Shannon entropy, ratio-based measures (e.g., digit-to-letter ratio), and suspicious keyword detection, enhancing feature richness. **Task 2:** Played an active role in data preprocessing by analyzing feature correlations, refining feature sets through Random Forest importance rankings, and preparing the dataset for tokenization. **Task 3:** Implemented and analyzed the performance of Extra Trees, SGD, and SVC classifiers, contributing to the overall evaluation of their strengths in detecting malicious URLs. **Task 4:** Conducted additional analysis of confusion matrices across all classifiers, identifying challenges in accurately classifying phishing attacks and proposing

	optimization strategies for underperforming models. **Task 5:** Drafted sections of the report focusing on problem statement, methods, and model analysis, ensuring coherence and clarity in presenting the project's objectives and findings. **Task 6:** Contributed to the conclusion and future work sections, summarizing the overall effectiveness of machine learning techniques and suggesting further improvements for handling imbalanced datasets.
Member 2: Haritha Injam	**Task 1:** Contributed to feature extraction by deriving structural attributes such as domain details, subdomain lengths, and top-level domain characteristics to provide a diverse feature set for analysis. **Task 2:** Participated in data preprocessing by encoding boolean variables, imputing missing values, and ensuring a balanced representation of features for model training. **Task 3:** Implemented machine learning models, including AdaBoost, Gaussian Naive Bayes, and SVC, and visualized their confusion matrices to evaluate performance across URL categories. **Task 4:** Assisted in analyzing and comparing the performance of models through detailed evaluation metrics, identifying strengths and limitations in classifying phishing and malware URLs. **Task 5:** Co-authored the model implementation and evaluation sections of the report, providing detailed descriptions of the methodology and outcomes for classifiers. **Task 6:** Collaborated on finalizing conclusions and recommendations, emphasizing improvements for classifier accuracy and robustness in handling imbalanced classes.
Member 3: Sai Aparanji Nemmani	**Task 1:** Conducted feature extraction by focusing on statistical characteristics such as URL length, domain entropy, and counts of special characters, ensuring comprehensive feature coverage. **Task 2:** Participated in data preprocessing by handling missing values, encoding categorical variables, and discretizing numerical features such as URL and domain lengths into categories to improve model performance. **Task 3:** Implemented and evaluated Decision Tree and Random Forest classifiers, generating confusion matrices and analyzing their contributions to accurately classifying benign and malicious URLs. **Task 4:** Contributed to the evaluation phase by designing performance metrics such as precision,

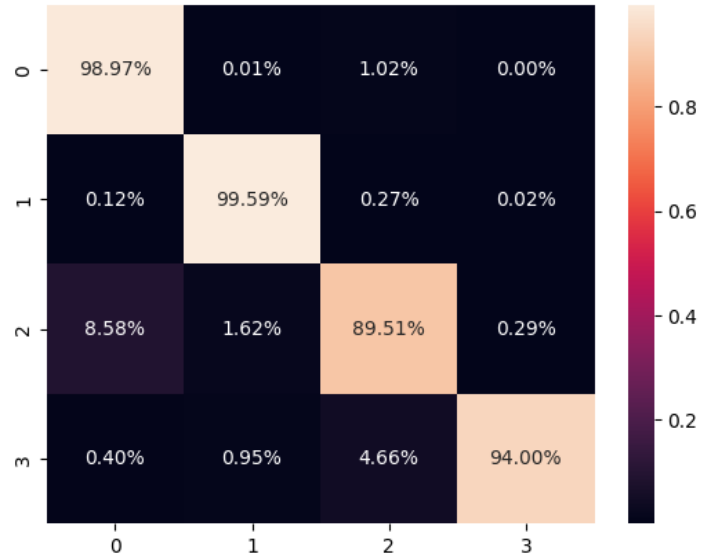
	recall, and F1 scores for the classifiers and analyzing the results to identify patterns in model behavior. **Task 5:** Played a key role in writing the dataset description, preprocessing, and feature selection sections of the final report, ensuring detailed documentation of these phases. **Task 6:** Provided insights into model optimization techniques and summarized findings for Decision Tree and Random Forest models in the result analysis section.

Appendix

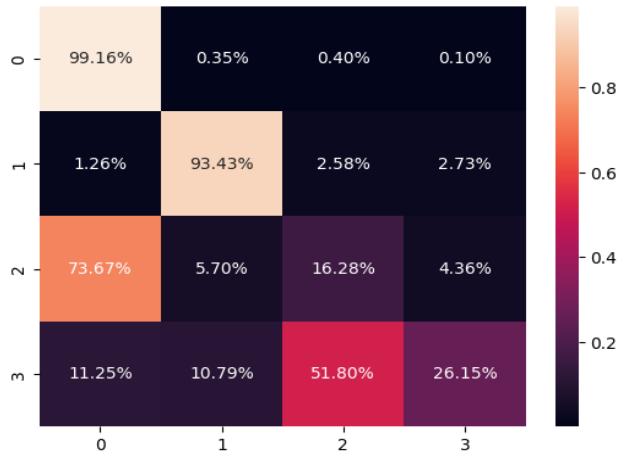
The confusion matrices of the models built by selecting all features are given below:



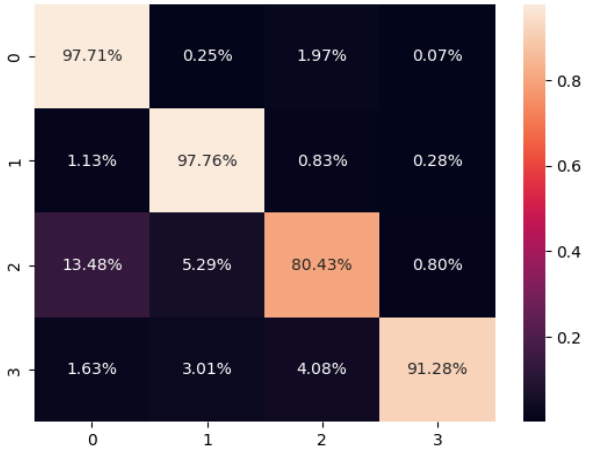
Confusion Matrix of Decision Trees



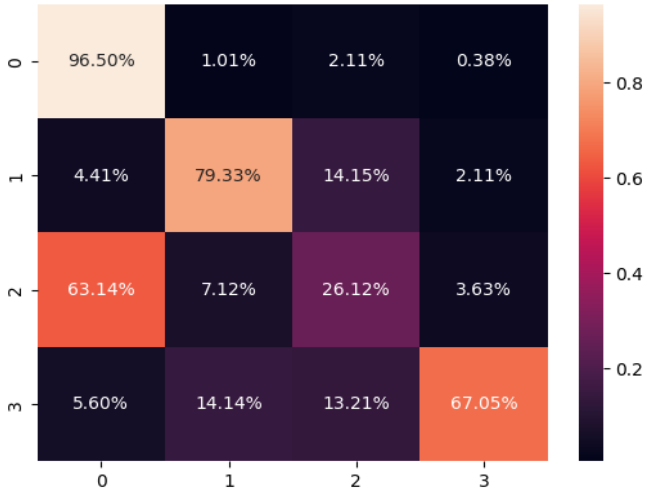
Confusion Matrix of Random Forest



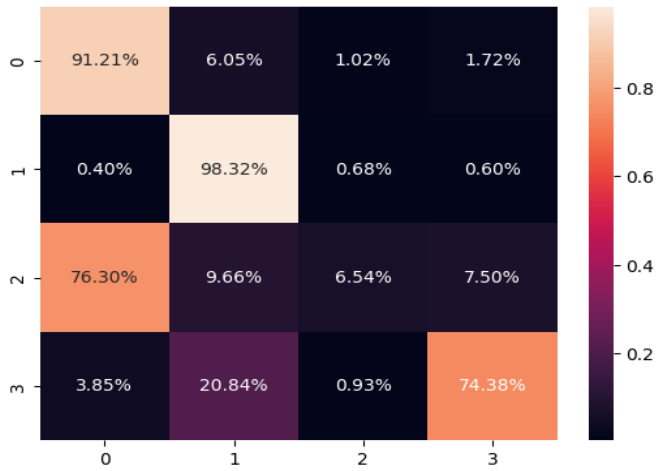
Confusion Matrix of AdaBoost Classifier



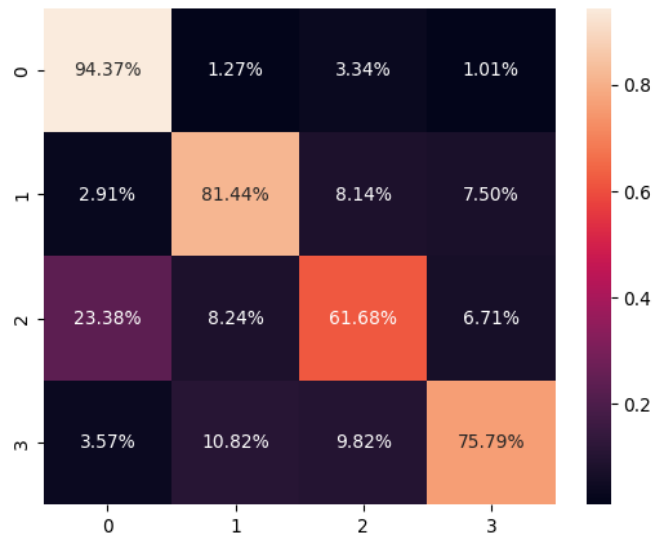
Confusion Matrix of KNN



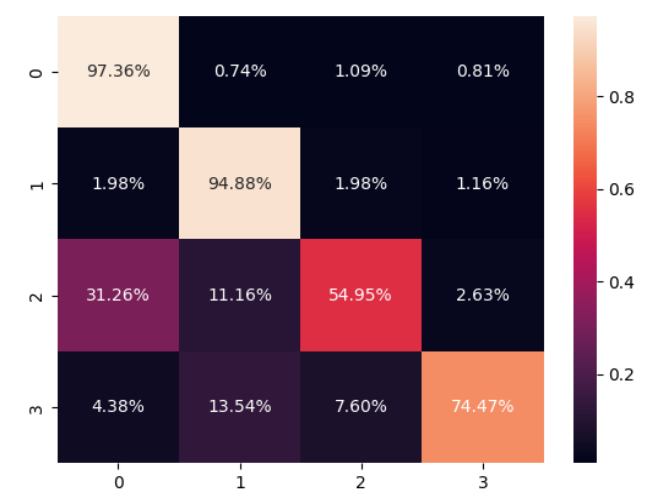
Confusion Matrix of Stochastic Gradient Classifier



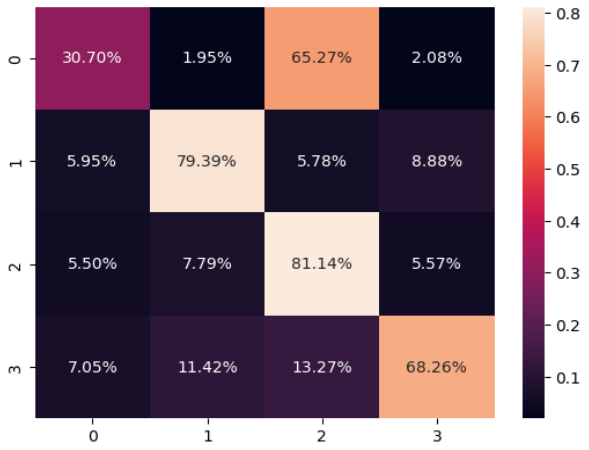
Confusion Matrix of Naïve Bayes



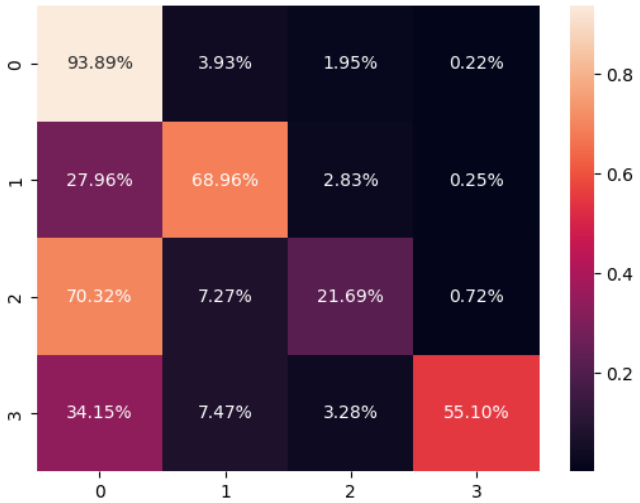
Confusion Matrix of Decision Trees



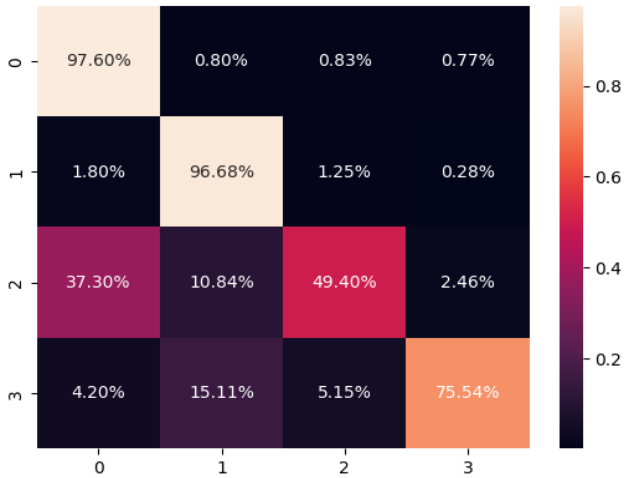
Confusion Matrix of Random Forest



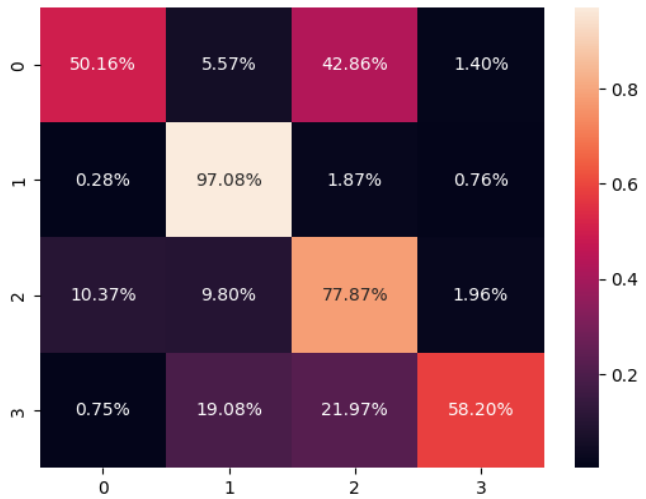
Confusion Matrix of AdaBoost Classifier



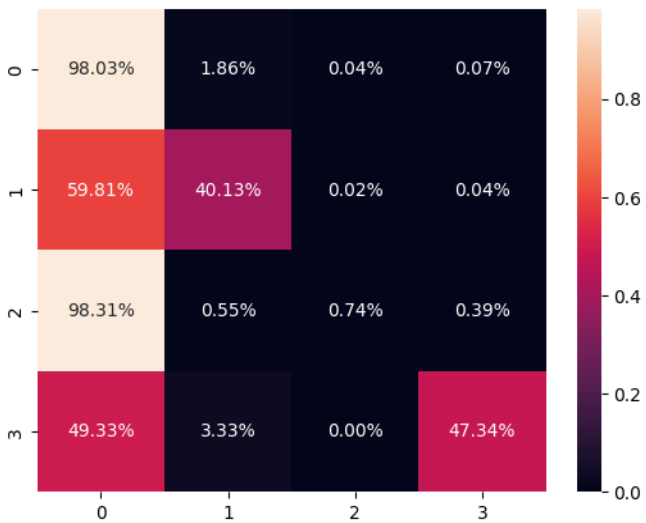
Confusion Matrix of KNN



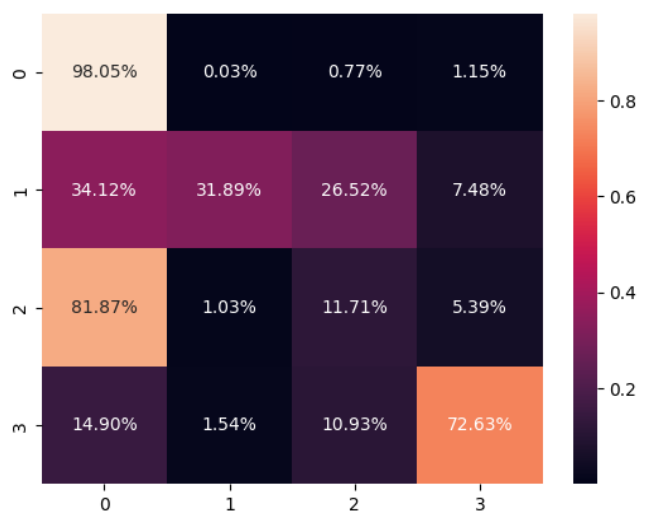
Confusion Matrix of Extra Tree Classifier



Confusion Matrix of Naïve Bayes

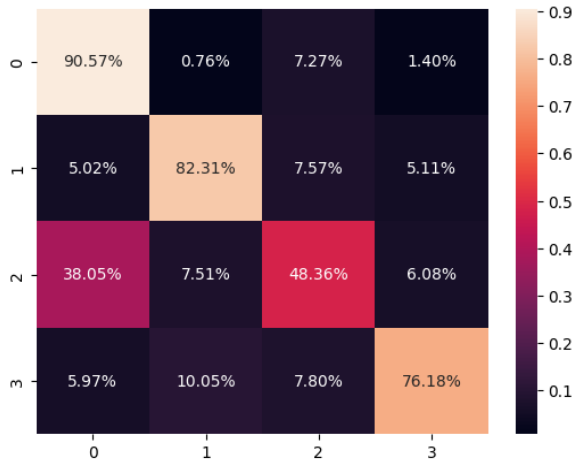


Confusion Matrix of Support Vector Classifier

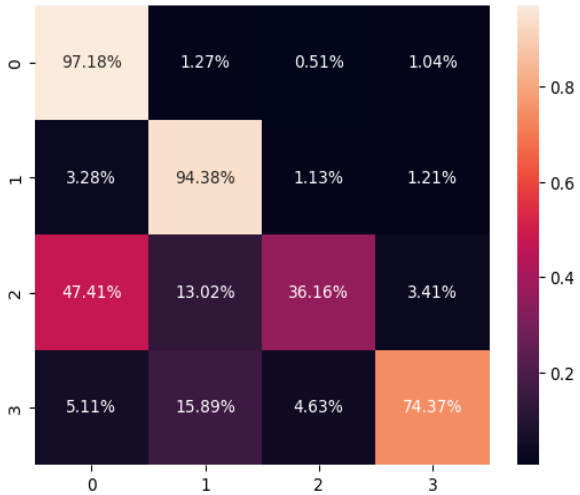


Confusion Matrix of Stochastic Gradient Classifier

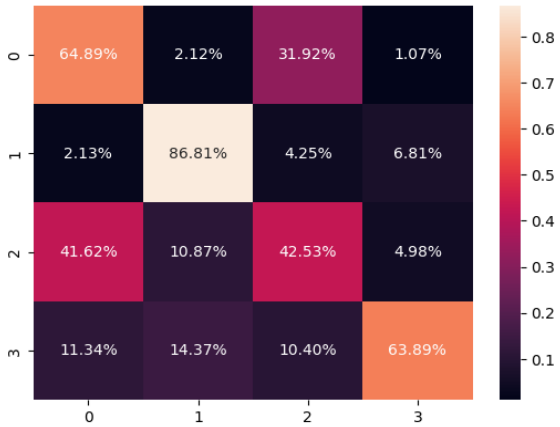
The confusion matrices of the models built by selecting features by correlation are given below:



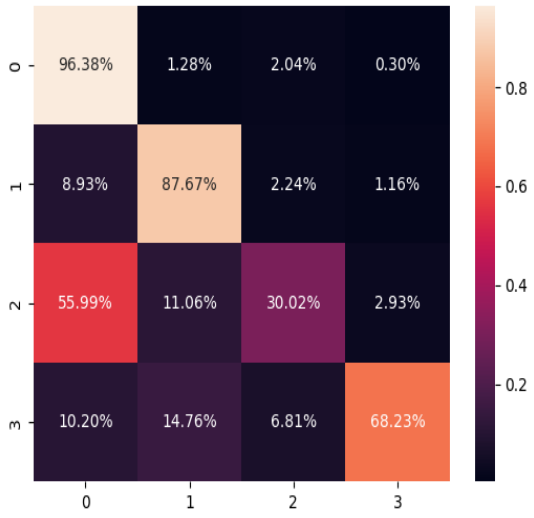
Confusion Matrix of Decision Trees



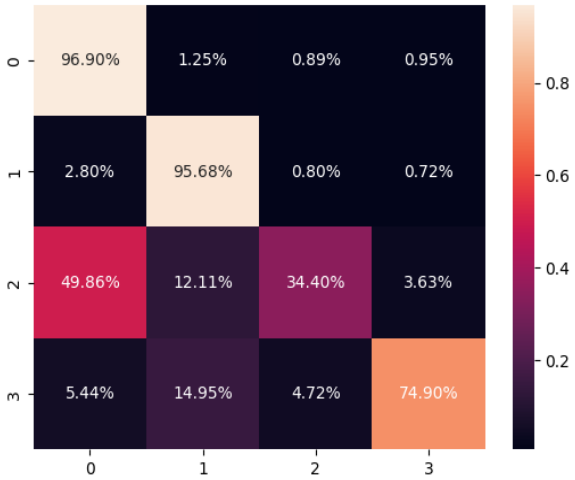
Confusion Matrix of Random Forest



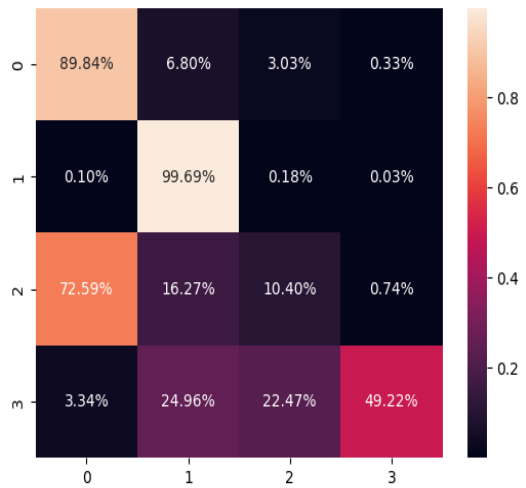
Confusion Matrix of AdaBoost Classifier



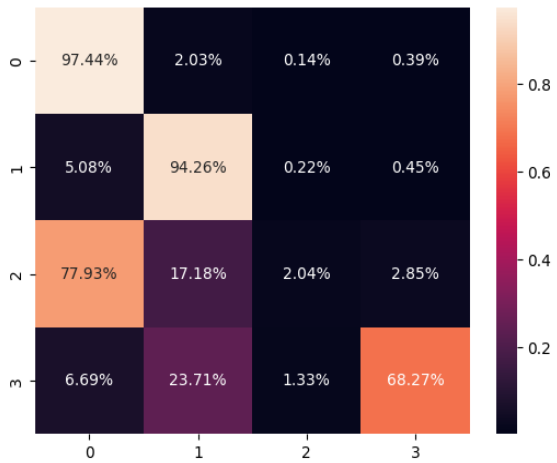
Confusion Matrix of KNN



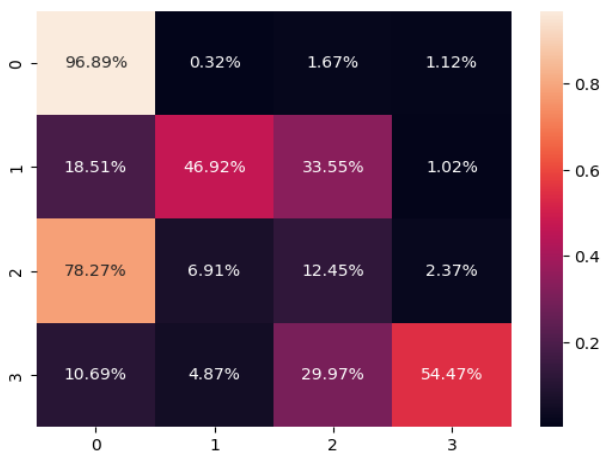
Confusion Matrix of Extra Tree Classifier



Confusion Matrix of Naïve Bayes

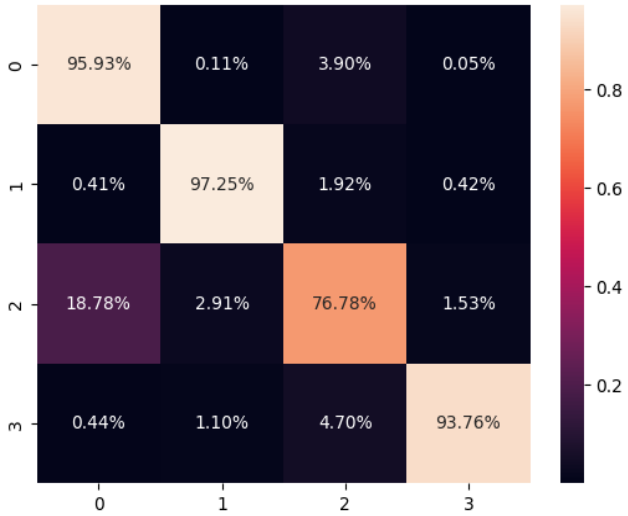


Confusion Matrix of Support Vector Classifier

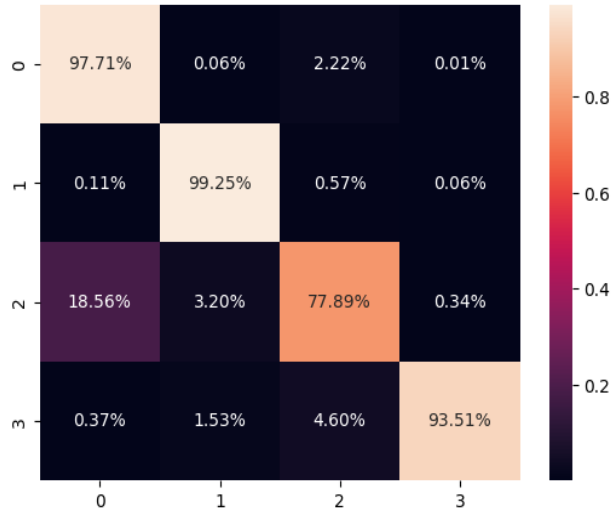


Confusion Matrix of Stochastic Gradient Classifier

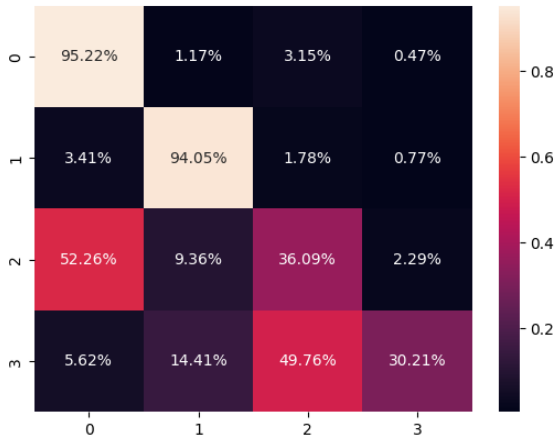
The confusion matrices of the models built by selecting features by random forest feature importance are given below:



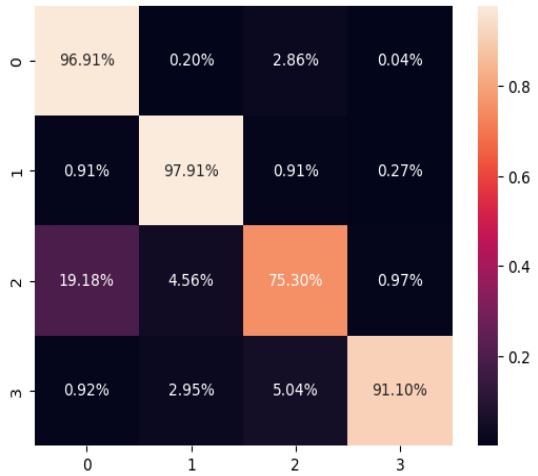
Confusion Matrix of Decision Trees



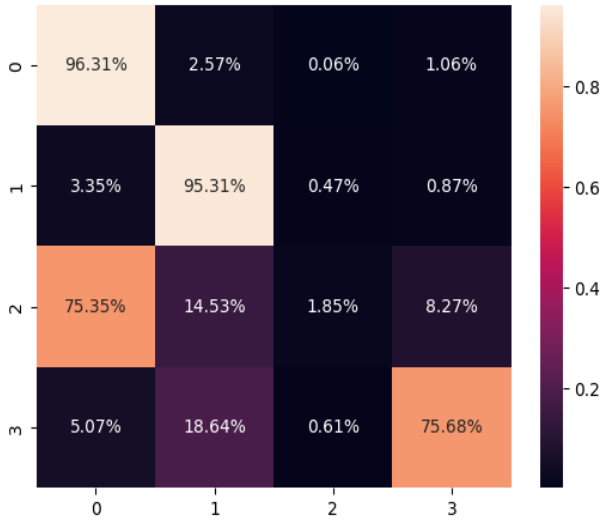
Confusion Matrix of Random Forest



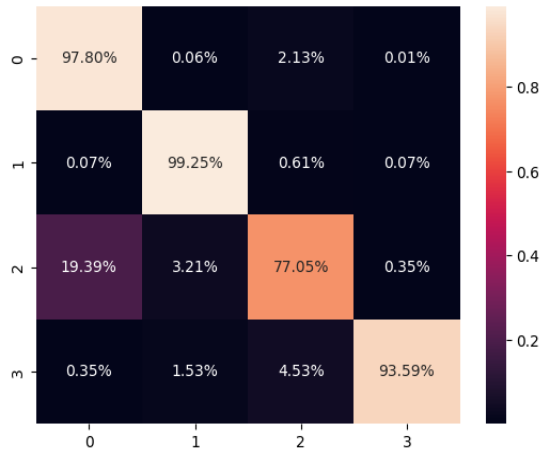
Confusion Matrix of AdaBoost Classifier



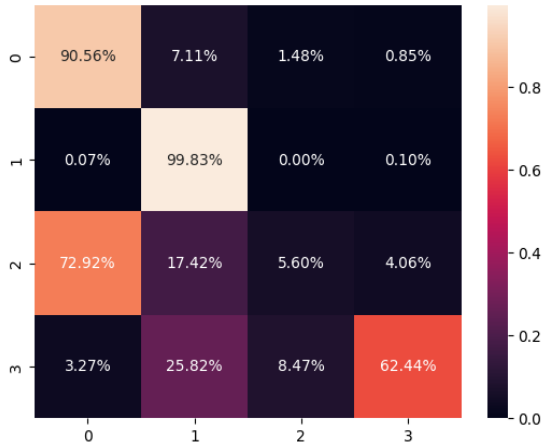
Confusion Matrix of KNN



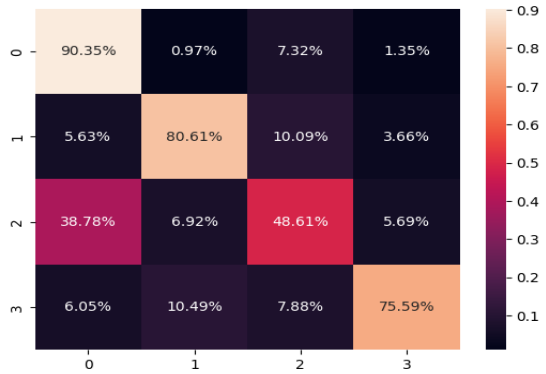
Confusion Matrix of Stochastic Gradient Classifier



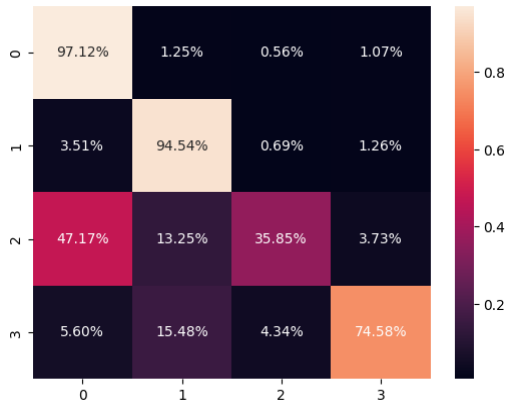
Confusion Matrix of Extra Tree Classifier



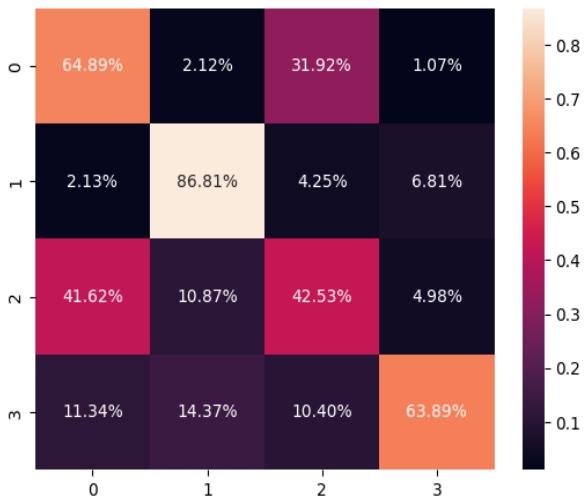
Confusion Matrix of Naïve Bayes



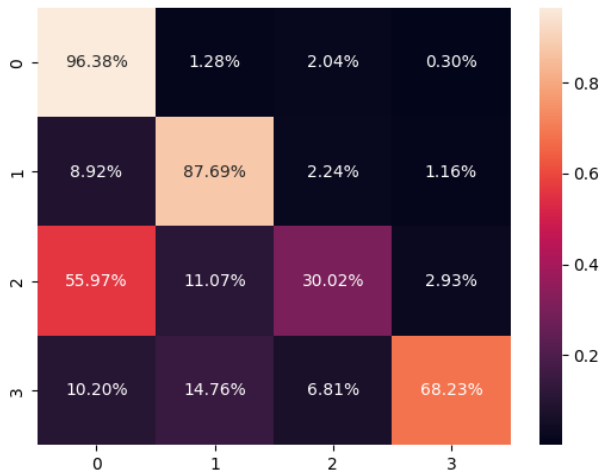
Confusion Matrix of Decision Trees



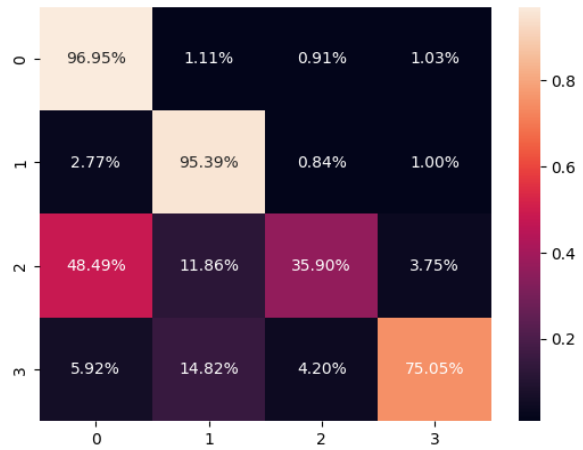
Confusion Matrix of Random Forest



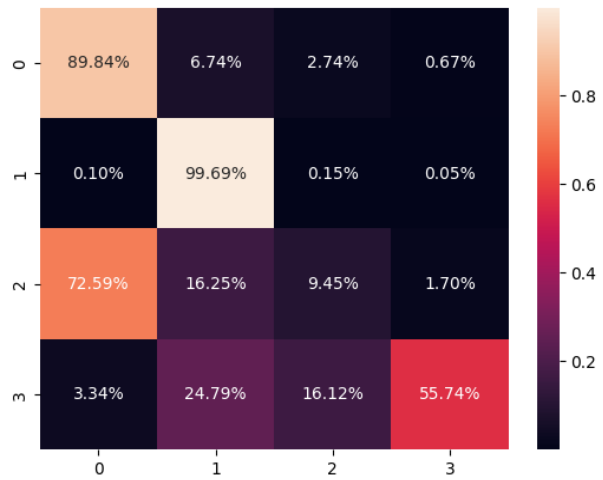
Confusion Matrix of AdaBoost Classifier



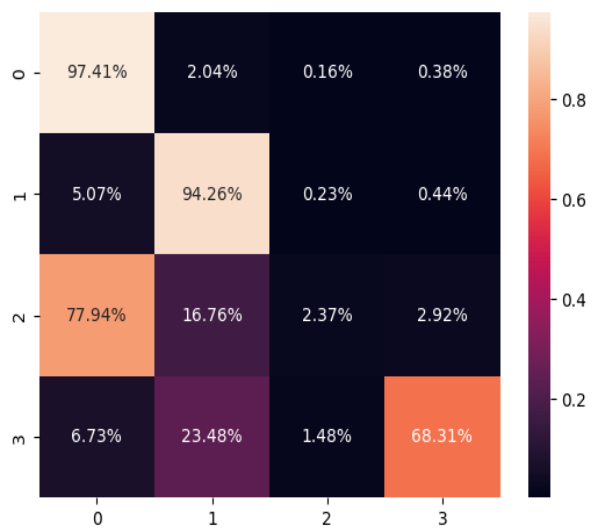
Confusion Matrix of KNN



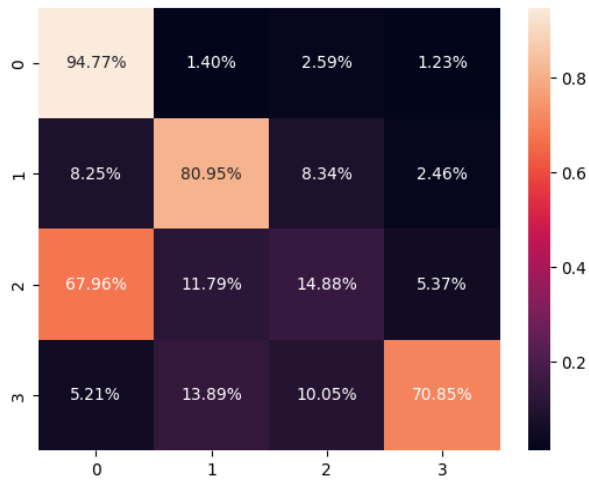
Confusion Matrix of Extra Tree Classifier



Confusion Matrix of Naïve Bayes

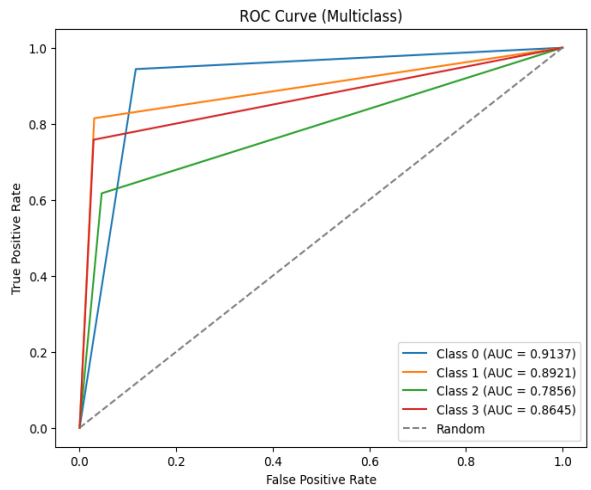


Confusion Matrix of Support Vector Classifier

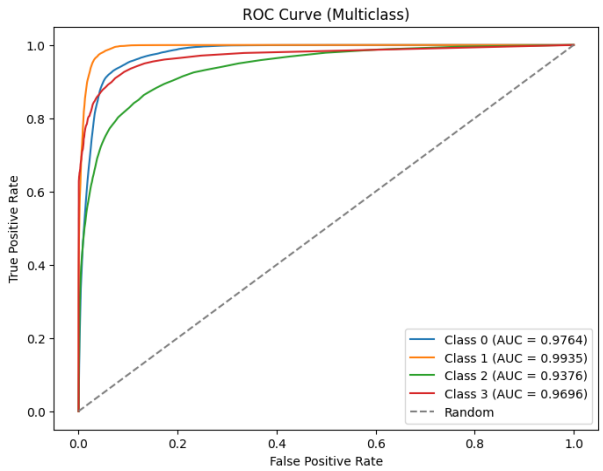


Confusion Matrix of Stochastic Gradient Classifier

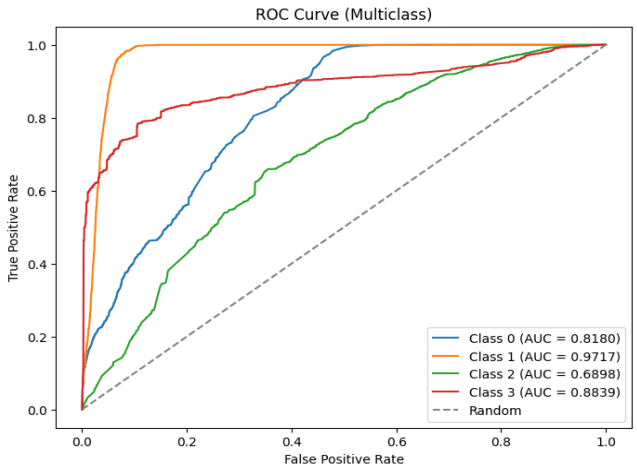
The ROC-AUC of the models built by selecting all features are given below:



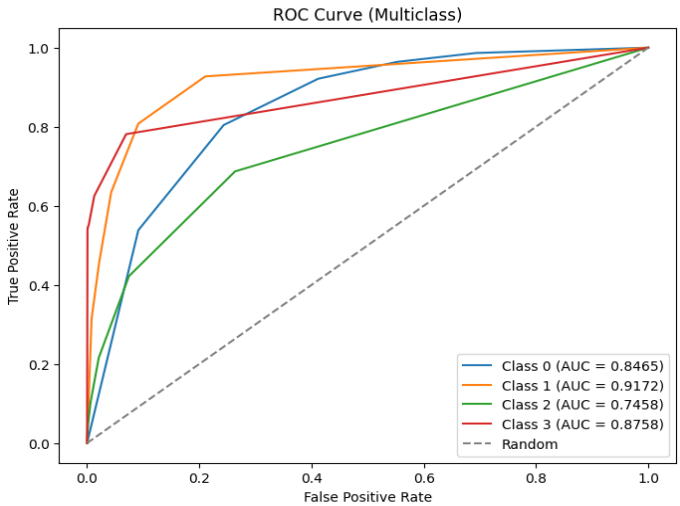
ROC of Decision Trees



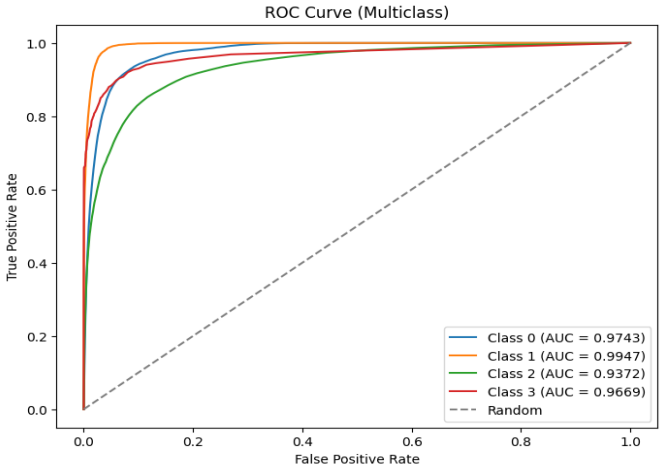
ROC of Random Forest Model



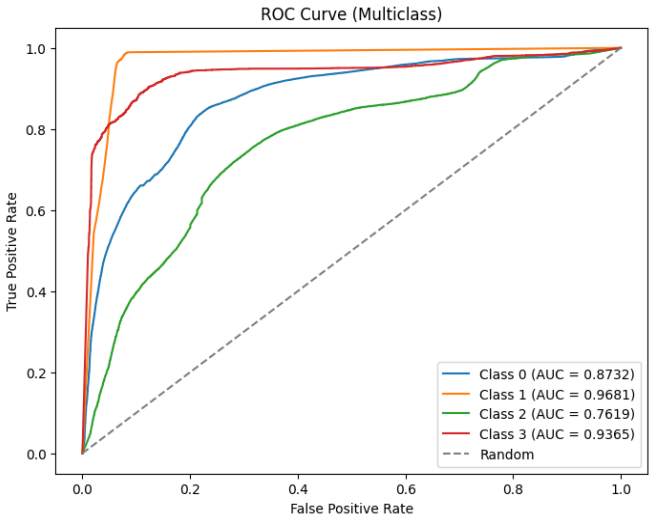
ROC of AdaBoost Classifier



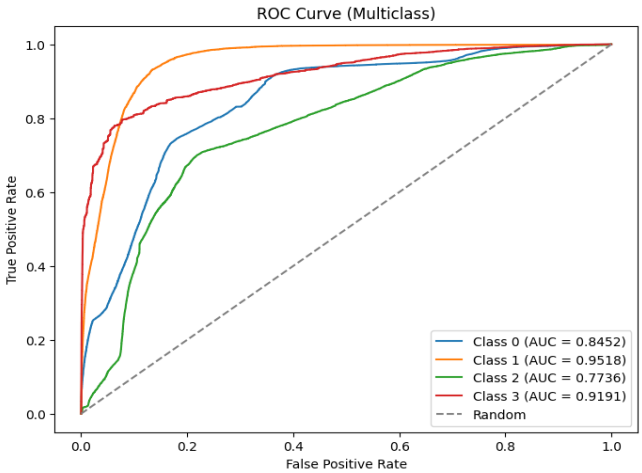
ROC of KNN



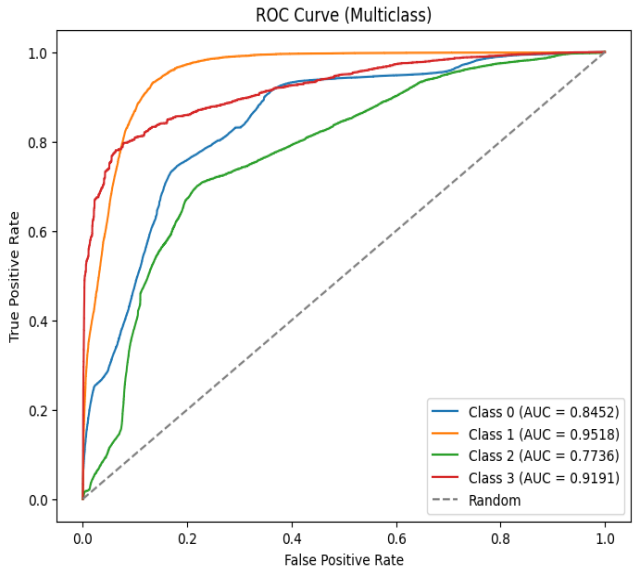
ROC of Extra Tree Classifier



ROC of Naïve Bayes

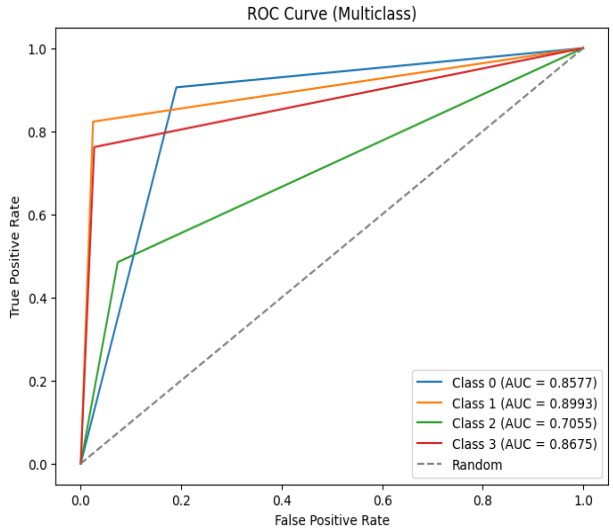


ROC of Support Vector Classifier

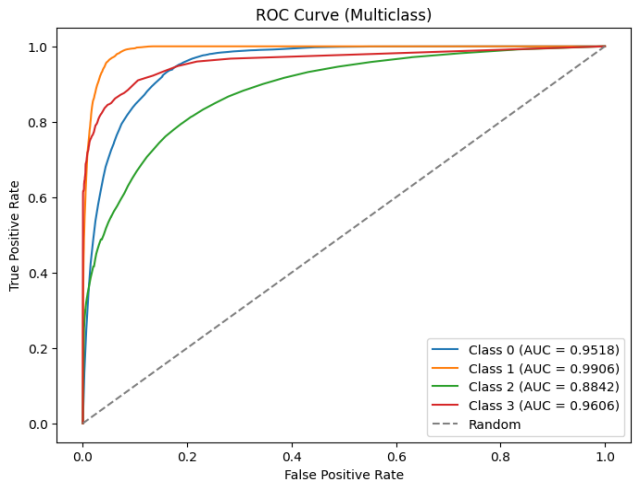


ROC of Stochastic Gradient Classifier

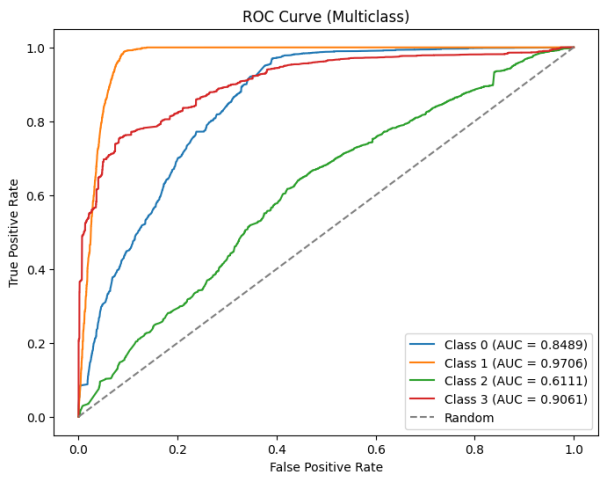
The ROC of the models built by selecting features by correlation are given below:



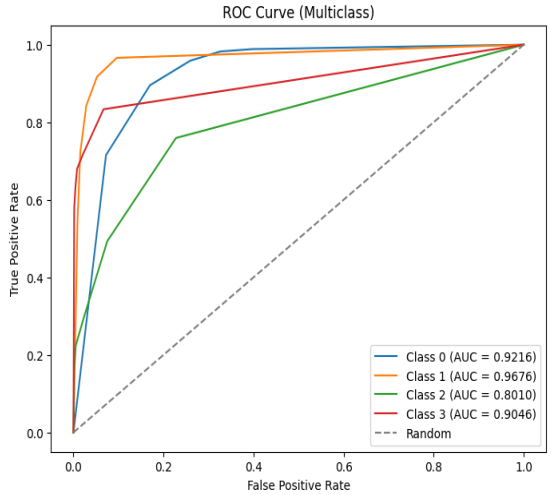
ROC of Decision Trees



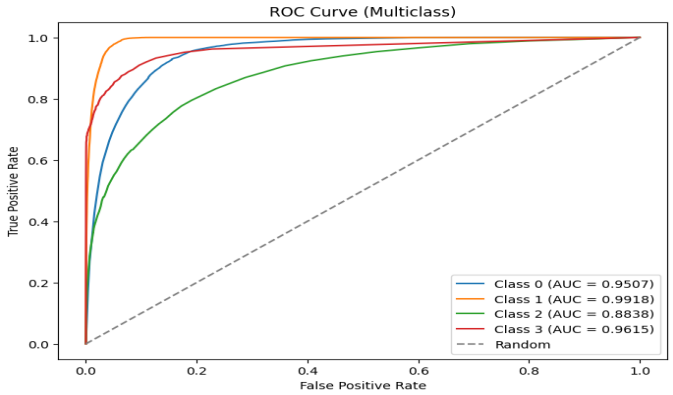
ROC of Random Forest



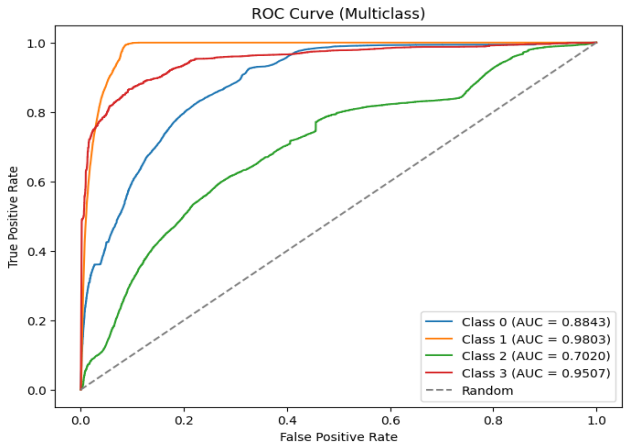
ROC of AdaBoost Classifier



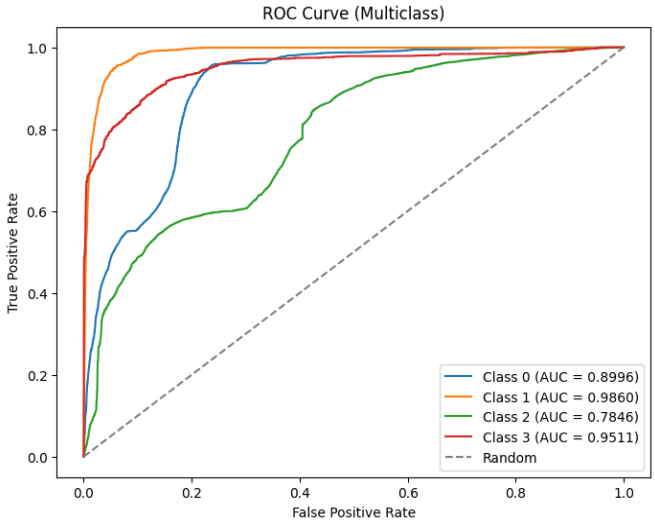
ROC of KNN



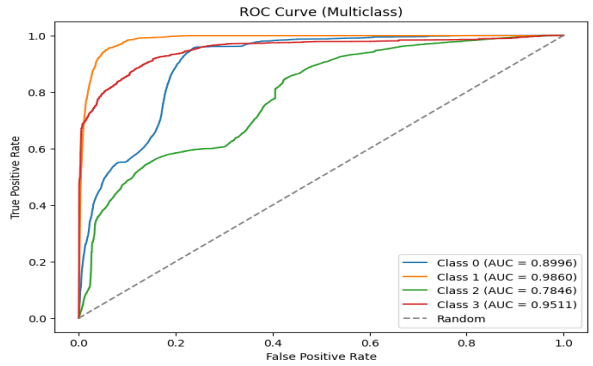
ROC of Extra Tree Classifier



ROC of Naïve Bayes

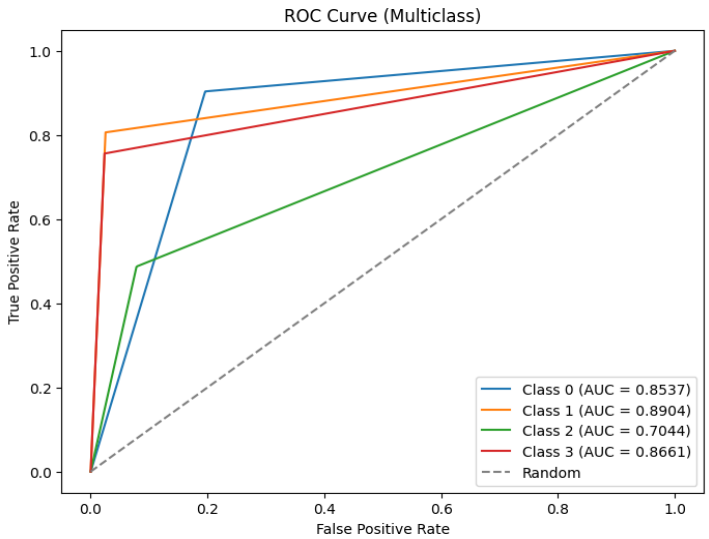


ROC of Support Vector Classifier

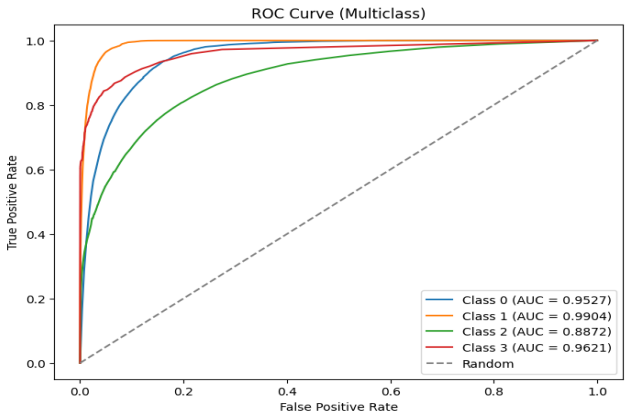


ROC of Stochastic Gradient Classifier

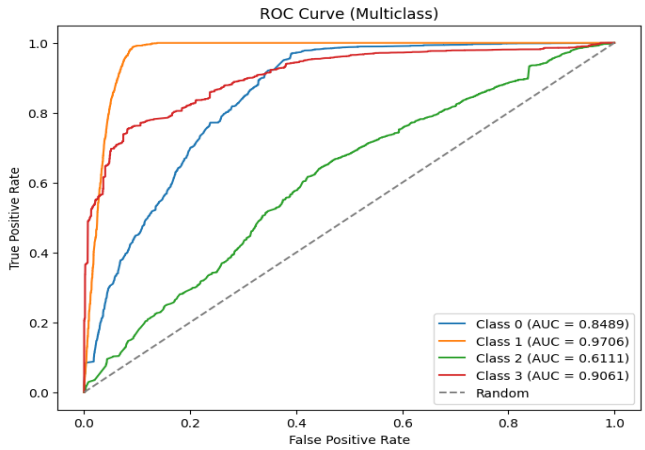
The ROC of the models built by selecting features by random forest feature importance are given below:



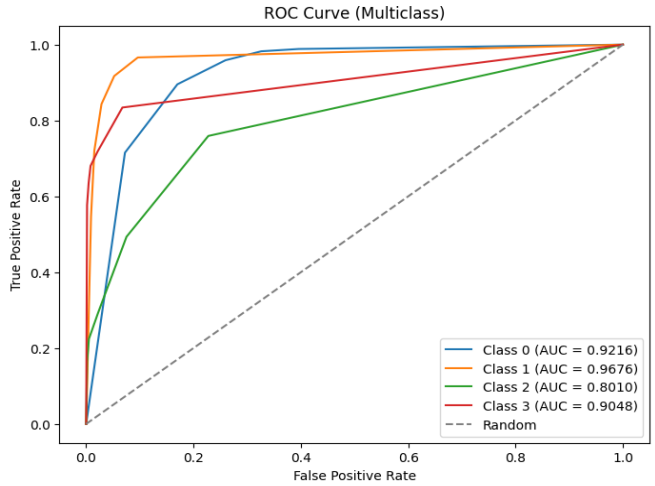
ROC of Decision Trees



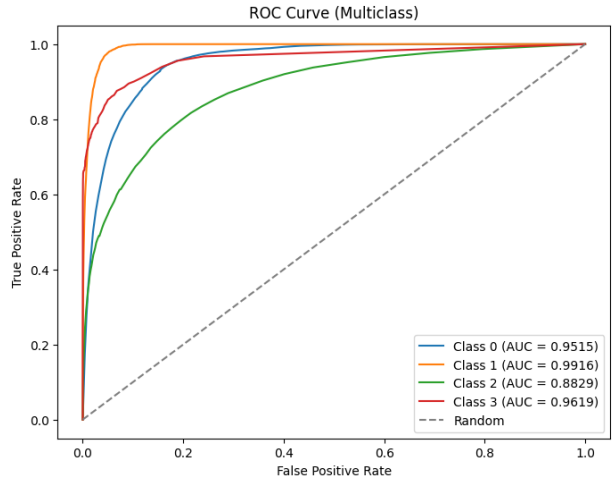
ROC of Random Forest Model



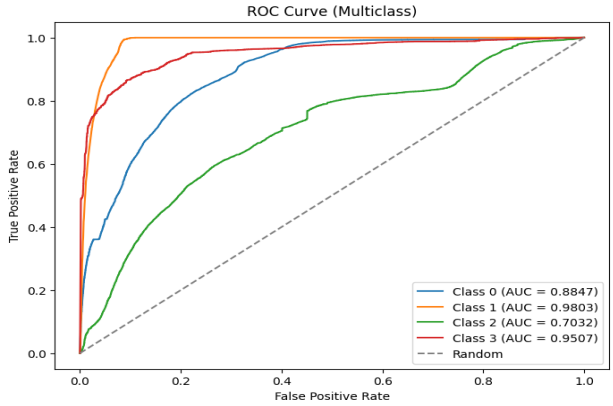
ROC of AdaBoost Classifier



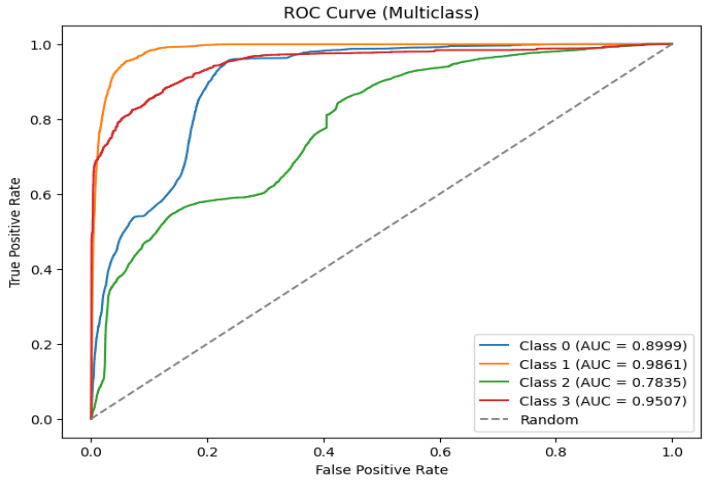
ROC of KNN



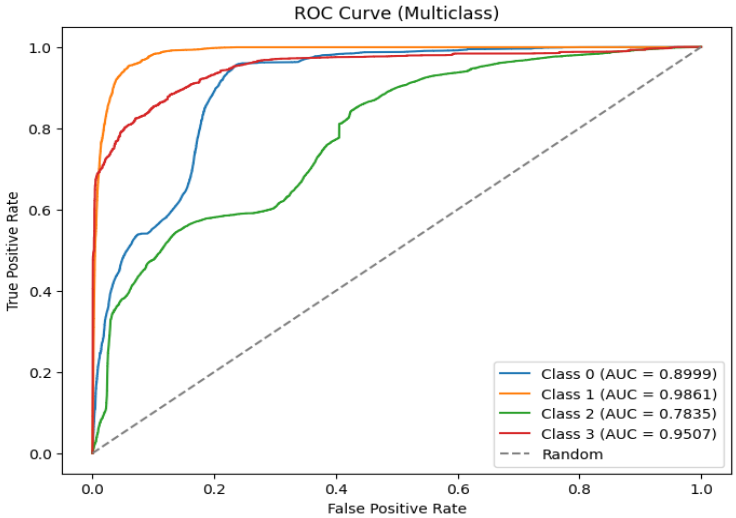
ROC of Extra Tree Classifier



ROC of Naïve Bayes



ROC of Support Vector Classifier



ROC of Stochastic Gradient Classifier